

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN
ENGLISH**

DIPLOMA THESIS

**Continuous Behavioral Biometrics for
Zero-Trust Network Access: An
Event-Driven Microservices
Architecture**

**Supervisor
Lect. Dr. Alina-Delia Călin**

*Author
Mihai Țălu*

2026

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ENGLEZĂ

LUCRARE DE LICENȚĂ

**Biometrie Comportamentală Continuă
pentru Accesul la Rețea de Tip
Zero-Trust: O Arhitectură de
Microservicii Bazată pe Evenimente**

**Conducător științific
Lect. Dr. Alina-Delia Călin**

*Absolvent
Mihai Țălu*

2026

ABSTRACT

Traditional static authentication mechanisms, such as passwords and one-time Multi-Factor Authentication (MFA), remain vulnerable to modern session hijacking and Adversary-in-the-Middle (AiTM) attacks because they only verify identity at the point of entry. To address this vulnerability, this thesis proposes a continuous authentication system utilizing behavioral biometrics (specifically free-text keystroke dynamics) embedded within a Zero-Trust Network Access (ZTNA) framework.

The primary contribution of this research is the engineering of a polyglot event-driven microservices architecture that bridges the gap between deep learning inference and web routing. By orchestrating a Java/Spring Boot API Gateway for WebSocket state management, a Redis Publish-Subscribe event broker, and a Python inference daemon, the system decouples tensor mathematics from network I/O.

Furthermore, to overcome the "Cold-Start" constraint of traditional multi-class algorithms, this research implements a PyTorch Siamese Neural Network optimized with Triplet Margin Loss. This approach achieves Few-Shot metric learning by projecting sequential typing rhythms into a 128-dimensional latent space, enabling the mathematical isolation of human identities the network has never explicitly seen before.

Empirical evaluation utilizing the Aalto University 136-million keystroke dataset indicates zero-shot verification efficacy within the identity anomaly detection domain. The proposed architecture yields a baseline Area Under the Curve (AUC) of 0.9106, which stabilizes at 0.9000 under hard negative mining. The resulting system is designed to detect and terminate compromised active sessions before malicious actions can be executed, providing Zero-Trust security while preserving the end-user experience.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.1.1	The Evolution of Remote Workspaces	2
1.1.2	Vulnerabilities in Static Authentication	2
1.2	Problem Statement	2
1.2.1	Session Hijacking and Insider Threats	2
1.2.2	Cold-Start and Free-Text Constraints	3
1.2.3	Gradual Mimicry and Template Poisoning	3
1.3	Proposed Solution	3
1.3.1	Zero-Trust Continuous Verification	3
1.3.2	Event-Driven Few-Shot Learning	3
1.4	Research Objectives	4
1.4.1	Decoupled Real-Time Metric Learning	4
1.4.2	Security Benchmarking	4
1.5	Scope and Limitations	4
1.6	Document Structure	4
1.7	Declaration of Generative AI and AI-assisted technologies in the writing process	5
2	Background and Literature Review	6
2.1	Behavioral Biometrics	6
2.1.1	Fundamentals of Keystroke Dynamics	6
2.1.2	Temporal Feature Extraction	7
2.1.3	Recent Advancements and the Deep Learning Paradigm	8
2.2	Machine Learning for Anomaly Detection	8
2.2.1	Limitations of Traditional Statistical Models	9
2.2.2	Siamese Networks and Metric Learning	9
2.2.3	Triplet Margin Loss and Paradigm Summary	9
2.3	Zero-Trust Architecture (ZTA)	10
2.3.1	Core Principles of Zero-Trust	10

2.3.2	Continuous Adaptive Risk and Trust Assessment	11
2.4	Real-Time Web Protocols	11
2.4.1	HTTP Polling vs. WebSockets	11
2.4.2	The STOMP Messaging Protocol	12
2.5	Distributed Systems and Middleware	12
2.5.1	Polyglot Microservices Architecture	12
2.5.2	Publish-Subscribe Patterns with Redis	12
3	System Architecture and Design	13
3.1	High-Level Architectural Overview	13
3.2	The Client-Side Application	15
3.2.1	Telemetry Capture and Sliding Windows	16
3.2.2	Dynamic UI State Management	16
3.3	The API Gateway Layer	17
3.3.1	WebSocket and STOMP Routing	18
3.3.2	Zero-Trust Security Middleware	18
3.4	The Event-Driven Message Broker	19
3.4.1	Redis Pub/Sub Topic Design	19
3.4.2	High-Speed Trust Score Caching	20
3.5	The Machine Learning Microservice	20
3.5.1	Daemon Architecture and Event Polling	20
3.5.2	The PyTorch Inference Pipeline	21
3.6	Data Persistence Strategy	23
3.6.1	In-Memory Biometric Persistence	24
4	Implementation and Data Engineering	25
4.1	Technology Stack Justification	25
4.2	Frontend Implementation	26
4.2.1	Engineering Keystroke Event Listeners	26
4.2.2	STOMP Client Integration and Telemetry Batching	27
4.3	Gateway Implementation	28
4.3.1	Configuring the Security Filter Chain	28
4.3.2	RedisTemplate Integration	29
4.3.3	Extending Zero-Trust to the REST API (Stateless Revocation)	29
4.4	ML Microservice Implementation	29
4.4.1	Data Engineering the Aalto 136M Dataset	29
4.4.2	Model Pre-Training with Triplet Loss	30
4.4.3	Deep Adaptive Profiling	31
4.5	Engineering Challenges Solved	31
4.5.1	Concurrency and Session Mapping	32

4.5.2	Metric Function Misalignment and Resolution	32
5	Evaluation	34
5.1	Experimental Setup and Methodology	34
5.1.1	Environment Specifications	34
5.1.2	Simulated Impostor Attack Methodology	35
5.1.3	Dataset Segregation and Zero-Shot Validation	35
5.2	Biometric Model Efficacy	36
5.2.1	Training Convergence Analysis	36
5.2.2	Discriminatory Power (ROC and AUC)	37
5.2.3	Genuine vs. Impostor Distributions	38
5.2.4	Model Interpretability via t-SNE Clustering	39
5.2.5	Algorithmic Stress Testing via Hard Negative Mining	39
5.2.6	Temporal Window Ablation Study	41
5.2.7	Cold-Start Calibration Window Justification	42
5.2.8	Threshold Sensitivity and Administrative Calibration	42
5.2.9	Adaptive Profiling and EMA Smoothing Factor (γ)	44
5.2.10	Comparative Analysis Against State-of-the-Art Baselines	45
5.2.11	Reconciliation of Biometric Efficacy Metrics	46
5.3	Architectural Efficiency and Theoretical Latency	47
5.3.1	Decoupled Stateful Inference	47
5.3.2	Constant-Time Revocation	47
6	Conclusion and Future Work	48
6.1	Summary of Contributions	48
6.2	Review of Research Objectives	48
6.3	System Limitations	49
6.4	Avenues for Future Research	51
	Bibliography	52

Chapter 1

Introduction

As technology keeps expanding at an unprecedented pace, security paradigms often get overlooked. Frequently, software engineering prioritizes rapid feature innovation over foundational security. Many modern software systems employ Multi-Factor Authentication (MFA) to strengthen user authentication. However, MFA generally verifies the user’s identity only during the initial login process, which may leave active sessions vulnerable to attacks that occur after authentication has been completed.

In this thesis, Zero-Trust Architecture and Behavioral Biometrics are combined in order to authenticate users in the background, continuously, without interrupting their workflows.

1.1 Context and Motivation

The foundation of digital security has historically relied on static authentication mechanisms, predominantly passwords. While these mechanisms verify the identity of a user at the exact moment of login, they do not guarantee the identity of the user for the duration of the active session.

Modern cyberattacks have evolved to exploit this vulnerability. Techniques such as Pass-the-Cookie attacks, Adversary-in-the-Middle (AitM) phishing, and session hijacking do not attempt to guess a user’s password; instead, they steal the cryptographic token (e.g., a JSON Web Token or session cookie) generated *after* a successful MFA challenge [Obs25]. Once an attacker injects this stolen token into their own browser, the backend server trusts the malicious connection until the token explicitly expires.

To mitigate this, the cybersecurity industry is migrating toward Zero-Trust Network Access (ZTNA). As defined by the National Institute of Standards and Technology (NIST) [RBMC20], Zero-Trust mandates that trust must never be implicitly

granted based on network location or past authentication. However, enforcing continuous authorization without destroying the user experience via constant MFA prompts remains a significant engineering challenge. Behavioral biometrics, specifically keystroke dynamics, offers a passive, invisible solution to continuous identity verification [NJ10].

1.1.1 The Evolution of Remote Workspaces

The global shift toward decentralized and remote work environments has dissolved the traditional corporate network perimeter [LSN⁺21]. Previously, organizations relied on secure physical office spaces and trusted local area networks (LANs) to shield internal applications. Today, sensitive corporate data is accessed from personal devices operating on untrusted public Wi-Fi networks. This paradigm shift means that perimeter defenses like VPNs are no longer sufficient; security must be enforced at the application and user identity layer, demanding constant vigilance over active sessions.

1.1.2 Vulnerabilities in Static Authentication

Static authentication acts as a single gatekeeper. Once the gate is passed, the system assumes the physical human at the keyboard remains the authorized user. This assumption can be exploited by modern threat actors utilizing malware, such as infostealers, which silently exfiltrate active session cookies from a compromised machine. Furthermore, the rise of "MFA Fatigue" attacks (where attackers spam a user's mobile device with approval requests until the frustrated user accidentally accepts) indicates that even secondary physical factors can be bypassed via human psychological manipulation [Wei22].

1.2 Problem Statement

1.2.1 Session Hijacking and Insider Threats

Despite the theoretical promise of continuous authentication, mitigating session hijacking in real-time introduces engineering bottlenecks. Traditional web architectures couple network I/O with business logic. Calculating biometric embeddings using Deep Learning requires heavy tensor mathematics that blocks standard HTTP threads, leading to high server latency under concurrent user load. If a continuous authentication system slows down a user's ability to type or interact with a webpage, it will be rejected by enterprise users.

1.2.2 Cold-Start and Free-Text Constraints

Furthermore, legacy biometric systems often utilize multi-class classifiers or anomaly detection algorithms (such as One-Class Support Vector Machines) [KM09]. These architectures require the system to be computationally retrained every time a new user is registered, and often require the user to type a specific, fixed phrase (e.g., a password) to evaluate their rhythm. This creates a “Cold-Start” problem, making them unscalable for modern dynamic applications where users type free-text across various UI elements.

1.2.3 Gradual Mimicry and Template Poisoning

Beyond instantaneous session hijacking, continuous biometric systems face the threat of “Gradual Mimicry.” In this scenario, a highly patient threat actor, or sophisticated malware, attempts to slowly alter their injected typing rhythm to mathematically approach the genuine user’s baseline. If the system’s adaptive profiling mechanisms are too permissive, the attacker can intentionally induce Concept Drift, slowly poisoning the biometric template over hours or days until their own malicious rhythm is fully trusted by the network. This necessitates an architecture that maintains mathematical inertia against adversarial template poisoning.

1.3 Proposed Solution

1.3.1 Zero-Trust Continuous Verification

To resolve these vulnerabilities, this research proposes a continuous, passive authentication pipeline embedded directly into the browser’s event loop. By capturing hardware-level micro-timings (traditionally defined in literature as Flight Time and Dwell Time [JG90]) and streaming them via WebSocket connections, the system enhances standard web applications into Zero-Trust environments. If the user steps away from their unlocked terminal and an unauthorized actor begins typing, the system mathematically detects the anomaly and physically revokes the JWT.

1.3.2 Event-Driven Few-Shot Learning

To overcome the Cold-Start constraint, the proposed architecture abandons traditional classifiers in favor of a Siamese Neural Network trained via Triplet Margin Loss [KZS15]. This model achieves few-shot metric learning, projecting sequential typing rhythms into a 128-dimensional latent space. This allows the system to identify human identities it has never seen before using purely free-text input. To

prevent computational bottlenecks, the deep learning inference is decoupled from the web layer utilizing a Redis Pub/Sub event broker.

1.4 Research Objectives

1.4.1 Decoupled Real-Time Metric Learning

The primary objective is to engineer a polyglot microservices architecture that bridges theoretical Deep Learning with practical software engineering. This involves orchestrating a Java/Spring Boot gateway for strict STOMP/WebSocket policy enforcement and a Python/PyTorch daemon restricted to mathematical inference.

1.4.2 Security Benchmarking

The secondary objective is to empirically evaluate the system against simulated session hijacking attacks using the Aalto University 136M keystroke dataset. The research aims to achieve an Area Under the Curve (AUC) demonstrating high discriminatory power against impostors, while the user experience remains unhindered.

1.5 Scope and Limitations

While the proposed architecture provides a robust continuous authentication framework, certain boundaries define the scope of this research. The biometric model is currently restricted to physical, mechanical keystroke dynamics captured via web browsers; it does not evaluate touchscreen dynamics (swipes, tap pressure) prevalent on mobile devices. Additionally, hardware variability (e.g., a user switching from a mechanical gaming keyboard to a membrane laptop keyboard mid-session) may temporarily elevate the False Rejection Rate (FRR). This cross-device variance is a widely recognized limitation in biometric literature [AMM⁺22], requiring the Exponential Moving Average (EMA) baseline to naturally adapt to the new hardware signature over time.

1.6 Document Structure

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and Literature Review):** Explores the historical evolution of keystroke dynamics, previous algorithmic approaches, and the foundational principles of Zero-Trust architecture.
- **Chapter 3 (System Architecture and Design):** Details the theoretical topology of the polyglot microservices, outlining the responsibilities of the Vue.js frontend, the Spring Boot Gateway, the Event Broker, the Python daemon and the Persistence layer.
- **Chapter 4 (Implementation and Data Engineering):** Provides a technical breakdown of the software engineering, including WebSocket state management and the mathematical sanitization of the Aalto 136M dataset.
- **Chapter 5 (Evaluation):** Presents the empirical results of the system under simulated attack, analyzing the mathematical efficacy of the model (ROC, AUC) and the temporal window.
- **Chapter 6 (Conclusion and Future Work):** Summarizes the core contributions of the research and proposes avenues for future optimization.

1.7 Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used Google Gemini in order to improve the language and readability of their thesis, for research and for code optimization. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Chapter 2

Background and Literature Review

This chapter establishes the theoretical and technological foundation for the proposed continuous authentication system. It explores the principles of behavioral biometrics, with a particular focus on keystroke dynamics, Zero-Trust Architecture, and event-driven microservices.

2.1 Behavioral Biometrics

Biometric authentication is broadly categorized into two distinct domains: physiological biometrics and behavioral biometrics. Physiological biometrics rely on static human characteristics, such as fingerprints, iris patterns or facial geometry. While these modalities offer high accuracy for one-time authentication (e.g., unlocking a smartphone), they are unsuitable for continuous verification, because requiring a user to continually stare into a camera or keep their finger on a scanner throughout an entire work session significantly degrades user experience.

In contrast, behavioral biometrics measure how a person acts rather than what they are. This includes gait analysis, mouse movement tracking, and keystroke dynamics. The primary advantage of behavioral biometrics in a remote workspace is its passive nature. It operates silently in the background, continuously analyzing the user's interaction with the system. By analyzing continuous streams of data, systems can achieve a state of persistent verification without requiring active user intervention, bridging the gap between high security and good user experience.

2.1.1 Fundamentals of Keystroke Dynamics

Keystroke Dynamics is a subset of behavioral biometrics that identifies users based on their typing rhythms. The fundamental premise of keystroke dynamics rests on neurophysiology and motor control. As an individual becomes proficient at typing, their cognitive processing and muscle memory synchronize to create highly

specific, repeatable reflex pathways [BW12]. Because these patterns are ingrained habits governed by the central nervous system, they are difficult for an attacker to consciously forge over a prolonged period.

By continuously analyzing these rhythms, a system can establish a baseline of trust that degrades the moment an anomaly happens (like a hijacker taking over the keyboard).

2.1.2 Temporal Feature Extraction

To translate typing behavior into quantifiable data for a machine learning model, raw keystroke events must be parsed into specific temporal features. The two most important features extracted in keystroke dynamics are:

Dwell Time (Hold Time): The duration a specific key is pressed. It is calculated as the time elapsed between the keydown event and the keyup event for a single key [KM09].

$$T_{dwell} = t_{keyup,i} - t_{keydown,i} \quad (2.1)$$

Flight Time (Latency): The duration between releasing one key and pressing the subsequent key. It is calculated as the time elapsed between the keyup event of the first key and the keydown event of the second key [KM09].

$$T_{flight(U-D)} = t_{keydown,i+1} - t_{keyup,i} \quad (2.2)$$

Alternatively, **Down-Down Flight Time**, which is highly effective for measuring overall typing cadence, is expressed as:

$$T_{flight(D-D)} = t_{keydown,i+1} - t_{keydown,i} \quad (2.3)$$

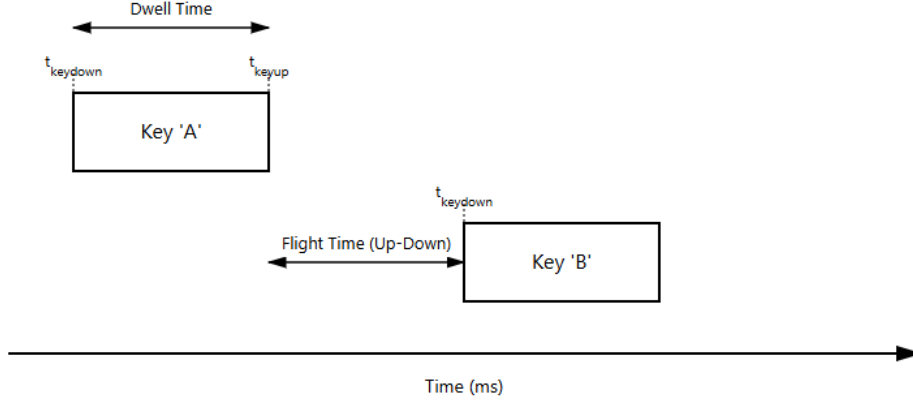


Figure 2.1: Visual representation of Dwell Time and Flight Time

2.1.3 Recent Advancements and the Deep Learning Paradigm

While early foundational studies established the viability of behavioral biometrics, the landscape of keystroke dynamics has undergone a paradigm shift in recent years due to the proliferation of artificial intelligence. In a comprehensive contemporary survey, Shadman et al. (2025) [SWM⁺25] emphasize that modern keystroke authentication has largely migrated away from classical statistical models (such as Gaussian Mixture Models and static Euclidean distance thresholds) toward deep representation learning.

The authors highlight that the current state-of-the-art relies on sophisticated feature engineering and neural architectures capable of capturing non-linear temporal dependencies. Specifically, the literature reveals a critical consensus: while traditional algorithms struggle with the high intra-user variance inherent in free-text typing, Deep Learning models (particularly Recurrent Neural Networks (RNNs) and Convolutional pipelines) excel at mapping these abstract timing vectors into highly discriminatory latent spaces [SWM⁺25].

It is upon this modern foundation that this thesis constructs its proposed architecture. By bypassing outdated statistical thresholds and instead adopting a Triplet Margin Loss-driven Siamese Network, this research directly addresses the contemporary challenges outlined in recent literature, pushing the boundaries of continuous Zero-Trust authentication using state-of-the-art spatial embeddings.

2.2 Machine Learning for Anomaly Detection

To enforce continuous validation, Machine Learning (ML) serves as the analytical engine, translating raw temporal data into mathematical identity verification.

2.2.1 Limitations of Traditional Statistical Models

Historically, keystroke dynamics relied on traditional statistical algorithms and shallow machine learning. Landmark studies, such as the evaluation by Killourhy and Maxion [KM09], utilized anomaly-detection algorithms like One-Class Support Vector Machines (OCSVM), k-Nearest Neighbors (k-NN), and Random Forests.

While effective in strictly controlled environments with “fixed-text” passwords, these models fail in modern “free-text” continuous authentication. Traditional models require extensive feature engineering (manually mapping n-grams and digraphs) and suffer from the “Cold-Start” problem, requiring hundreds of keystrokes from a single user to train a dedicated, user-specific classifier. In a dynamic web environment, retraining a unique SVM for every new user is computationally unscalable.

2.2.2 Siamese Networks and Metric Learning

To overcome the limitations of traditional algorithms, the State-of-the-Art (SOTA) in behavioral biometrics has shifted toward Deep Metric Learning, specifically utilizing Siamese Neural Networks [WHC⁺25]. Originally introduced for signature verification, Siamese architectures consist of twin neural networks with shared weights.

Instead of classifying a user into a predefined category, a Siamese Network learns a similarity function. Recent SOTA models, such as TypeNet [AMM⁺22], utilize Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) inside a Siamese structure to project raw keystroke sequences into a high-dimensional, latent spatial embedding. By measuring the Euclidean or Manhattan distance between two embeddings, the system can perform *Few-Shot* authentication, accurately verifying users it has never explicitly been trained on.

2.2.3 Triplet Margin Loss and Paradigm Summary

To train a Siamese Network to differentiate human typing rhythms, the standard cross-entropy loss is insufficient. Instead, the network is optimized using Triplet Margin Loss.

During training, the model evaluates three samples simultaneously: an Anchor (A), a Positive sample (P) from the same user, and a Negative sample (N) from a different user. The objective function minimizes the distance between the Anchor and Positive, while maximizing the distance between the Anchor and Negative by a predefined margin (α).

The loss function is mathematically defined as:

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|_2 - \|f(A) - f(N)\|_2 + \alpha, 0)$$

By aggressively penalizing the network when an impostor’s rhythm encroaches on a genuine user’s latent space, the model learns universal biomechanical features, enabling robust Zero-Shot anomaly detection in live enterprise environments [STV⁺23].

2.3 Zero-Trust Architecture (ZTA)

The integration of advanced biometric ML models provides the mathematical capability for continuous authentication, but these models must be orchestrated within a rigid cybersecurity framework. Zero-Trust Architecture represents this paradigm shift.

2.3.1 Core Principles of Zero-Trust

Coined by John Kindervag during his tenure at Forrester Research in 2010 [Kin10], and later formalized by organizations like NIST, the core philosophy of Zero-Trust is “never trust, always verify”. According to NIST Special Publication 800-207 [RBMC20], Zero-Trust is built on three foundational pillars:

1. **Verify Explicitly:** Authentication must continuously utilize all available data points, including user identity, location, and behavioral anomalies.
2. **Use Least Privileged Access:** Users are granted only the minimum access required to perform a task, reducing the attack surface.
3. **Assume Breach:** The network operates under the assumption that threat actors are already present inside the environment.

To illustrate why this paradigm is necessary, Table 2.1 contrasts legacy perimeter security with modern Zero-Trust methodologies.

Feature	Traditional Perimeter Security	Zero-Trust Architecture
Trust Model	Implicit trust after initial login	"Never trust, always verify."
Authentication	Static (Passwords, one-time MFA)	Continuous and adaptive
Session Hijacking	Vulnerable once the token is stolen	Mitigated via behavioral anomalies

Table 2.1: Traditional Security vs. Zero-Trust Architecture

2.3.2 Continuous Adaptive Risk and Trust Assessment

Within ZTA, the static binary "authenticated or unauthenticated" state is replaced by Continuous Adaptive Risk and Trust Assessment (CARTA) [SSS⁺22]. Keystroke dynamics perfectly fulfill the CARTA requirement by acting as a passive telemetry stream. As long as the calculated Euclidean distance of the user's typing rhythm remains below the anomaly threshold, their active Session JWT is considered valid. If a session hijacking occurs, the typing rhythm changes, the trust score degrades, and the system autonomously severs the connection in accordance with CARTA principles.

2.4 Real-Time Web Protocols

For continuous authentication to function without degrading user experience, the system requires low latency data transmission protocols.

2.4.1 HTTP Polling vs. WebSockets

Traditional web architectures utilize HTTP/1.1 or HTTP/2, which are fundamentally stateless and uni-directional. Attempting to transmit continuous keystroke telemetry via HTTP Short-Polling introduces prohibitive overhead, as every transmission requires a full DNS lookup, TCP handshake, and TLS negotiation.

Conversely, the WebSocket protocol (RFC 6455) provides a stateful, full-duplex communication channel over a single TCP connection. By utilizing WebSockets, the frontend sensor can stream micro-batches of keystrokes to the server with low network latency.

2.4.2 The STOMP Messaging Protocol

While WebSockets provide the transport layer, they lack native routing semantics. To establish a structured Publish-Subscribe (Pub/Sub) architecture between the client and the API Gateway, the Simple Text Oriented Messaging Protocol (STOMP) is utilized as a sub-protocol. STOMP allows the system to route telemetry to specific queues (e.g., `/app/stream-biometrics`) and subscribe the client to secure enforcement topics (e.g., `/queue/trust-score`).

2.5 Distributed Systems and Middleware

The final architectural requirement is ensuring that the computationally expensive Machine Learning inference does not bottleneck the high-frequency I/O of the web server.

2.5.1 Polyglot Microservices Architecture

The system eschews monolithic design in favor of a Polyglot Microservices Architecture [DGL⁺17]. This paradigm allows distinct layers of the application to be decoupled and written in the programming language best suited for its specific workload. A Java/Spring Boot Gateway handles the concurrent WebSocket connections and Zero-Trust policy enforcement, while an isolated Python daemon handles the tensor calculations required by the PyTorch Siamese Network.

2.5.2 Publish-Subscribe Patterns with Redis

Synchronous communication (e.g., REST API calls) between the Java Gateway and the Python ML Engine would cause thread-blocking, leading to system failure under load. To enforce decoupling, an in-memory data store, Redis, is deployed as an event broker.

Using Redis Pub/Sub, the Java gateway fires sanitized JSON telemetry into a dedicated ingress topic (`biometric_stream`) and releases the HTTP thread. The Python daemon consumes the event asynchronously, evaluates the biometric score using the PyTorch model, and publishes the resulting pass/fail metric back to an egress topic (`auth_results`). The Java gateway asynchronously listens to this topic via a `MessageListenerAdapter` and executes the final session revocation if necessary.

Chapter 3

System Architecture and Design

The primary objective of this system is to bridge the gap between theoretical Zero-Trust Network Access (ZTNA) [RBMC20] and practical, real-time software engineering. This chapter details the architecture required to support a continuous biometric authentication pipeline while maintaining low latency and preserving the end-user experience.

3.1 High-Level Architectural Overview

Continuous behavioral biometrics introduces a distributed systems challenge: the system must process telemetry data (keystrokes) and perform inference without blocking the primary application thread.

To satisfy these constraints, the proposed architecture abandons traditional monolithic paradigms in favor of a **Polyglot Event-Driven Microservices Architecture** [New21]. The system is divided into four distinct layers, each utilizing a specialized technology stack suited for its specific workload:

1. **The Telemetry Sensor (Frontend):** A Single Page Application (SPA) responsible for capturing raw keystroke data and managing local state.
2. **The Ingestion Gateway (Middleware):** A Java/Spring Boot server handling secure WebSocket (STOMP) connections, acting as the Zero-Trust policy enforcement point.
3. **The Event Broker:** A Redis Pub/Sub deployment providing high-throughput, volatile message routing between the gateway and the ML engine.
4. **The Biometric Evaluation Engine:** An isolated Python daemon running a pre-trained PyTorch Siamese Neural Network, restricted to mathematical inference.

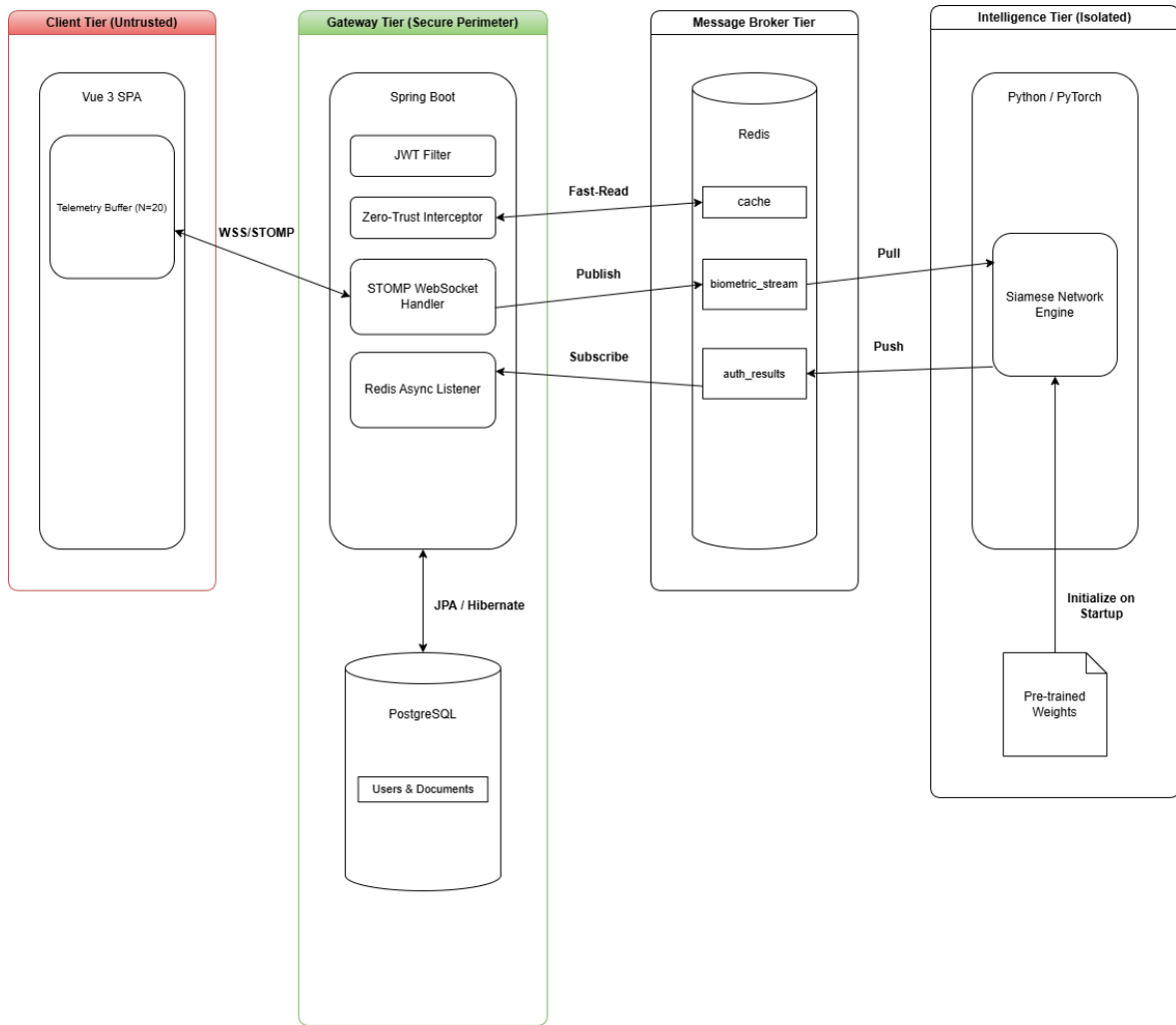


Figure 3.1: High-Level Structural Architecture. The diagram illustrates the isolation between the untrusted client sensor, the secure API gateway, and the Machine Learning evaluation engine.

As illustrated in Figure 3.1, this layered approach prevents tight coupling. The integration of the Redis Event Broker allows the Java gateway to release HTTP threads back to the pool after receiving telemetry.

Once the physical structure is established, the temporal execution of these components becomes critical. Figure 3.2 details the sequential flow of data across these layers during both normal operational profiling and a simulated session hijacking attack.

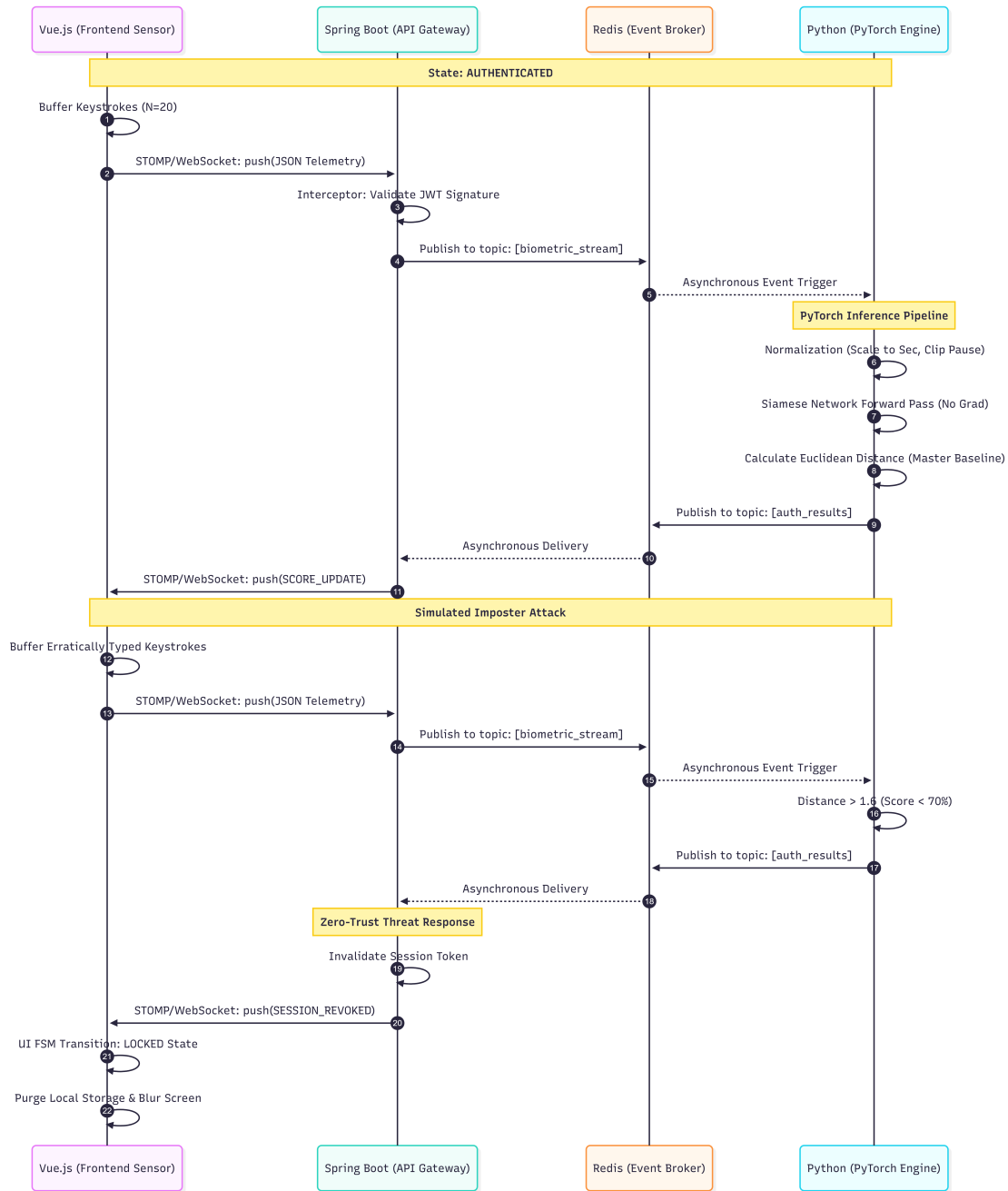


Figure 3.2: UML Sequence Diagram illustrating the decoupled Zero-Trust biometric evaluation pipeline during both normal operation and a simulated imposter attack.

This decoupling helps prevent neural network forward passes from blocking the I/O-bound web server handling concurrent WebSocket connections.

3.2 The Client-Side Application

In a Zero-Trust architecture, the client-side application is inherently treated as a

compromised environment [RBMC20]. Therefore, the frontend is explicitly stripped of any biometric evaluation capabilities. It functions solely as a “passive sensor”, passing raw chronological telemetry to the secure backend.

3.2.1 Telemetry Capture and Sliding Windows

The accuracy of a Siamese Neural Network is reliant on the precision of the input timestamps. Standard browser event listeners (`keyup` and `keydown`) are captured asynchronously. To facilitate microsecond precision, the system avoids standard UNIX epoch timestamps, which are susceptible to local system clock drift.

Instead, the frontend captures the raw timestamps from which the following primary temporal features are defined (the actual extraction is performed by the Python ML engine):

- **Dwell Time (DT):** The duration a physical key remains depressed. Mathematically defined as $DT_i = R_i - P_i$, where R is the release timestamp and P is the press timestamp of the i -th key.
- **Flight Time (FT):** The latency between consecutive keystrokes. While the literature defines multiple flight time variants (as introduced in Section 2.1.2), this architecture explicitly implements Down-Down (D-D) Flight Time, mathematically defined as $FT_i = P_{i+1} - P_i$. The D-D variant was selected over the Up-Down (U-D) variant because it provides a more robust measurement of overall typing cadence. Furthermore, during rapid typing, users frequently press a subsequent key before releasing the previous one (a biomechanical overlap known as “finger rolling”). This overlap causes U-D Flight Time to yield mathematically negative values. Utilizing D-D Flight Time yields a positive temporal sequence, simplifying the tensor normalization pipeline for the Siamese Network.

To optimize network bandwidth and prevent WebSocket flooding, the telemetry is buffered locally using a chronological sliding window array. The client aggregates sequential asynchronous typing interactions, transmitting the payload to the backend infrastructure immediately upon reaching a predefined threshold of 40 raw discrete events. Rather than enforcing synchronous pairing at the unauthenticated client layer, this event volume establishes a stochastic upper bound targeting an operational sequence length of $N = 20$ keystrokes, leaving the structural zero-padding to be executed deterministically by the backend inference pipeline.

3.2.2 Dynamic UI State Management

Because continuous authentication happens invisibly in the background, the User

Interface (UI) must gracefully handle asynchronous trust revocations. The frontend implements a strict finite state machine (FSM) governed by the incoming WebSocket messages from the Spring Boot gateway:

- **PROFILING (Cold Start):** The UI allows input but displays a visual indicator that the system is currently calibrating. To enable mathematical stability and prevent outlier corruption (e.g., a single typo), the Python inference engine requires an empirical baseline of 5 consecutive telemetry batches. This window accumulates a total of 200 raw discrete UI events; after backend boundary reconciliation and pairing filtration, this volume maps to a theoretical upper bound of at most $N = 100$ fully paired physical keystrokes. The UI remains strictly in this Read-Only profiling state until the 5th batch is processed and the averaged 128-dimensional Master Baseline is stored into the Redis cache.
- **AUTHENTICATED:** The standard operational state. The user has full interaction privileges as long as their rolling Trust Score remains above the administrative threshold.
- **LOCKED:** Triggered via an asynchronous server push if the Trust Score falls below 70%. The frontend immediately disables the document text area and updates the visual trust indicator to an error state, enforcing the Zero-Trust access control.

3.3 The API Gateway Layer

In standard web architectures, the backend server handles both HTTP request routing and business logic execution. However, in this continuous biometric system, the Java/Spring Boot application acts as an **Ingestion Gateway**. Its primary responsibility is to maintain persistent, stateful connections, sanitize incoming telemetry, and act as the definitive Policy Enforcement Point (PEP) for the Zero-Trust architecture.

To establish the baseline perimeter security, the system implements a standard registration and authentication flow. Upon a successful login challenge, the Spring Boot gateway issues a static JSON Web Token (JWT) to the client. However, because this static token represents a single point of failure (vulnerable to session hijacking and Pass-the-Cookie attacks), the system immediately transitions from this traditional perimeter defense to the continuous biometric Zero-Trust engine for the duration of the active session.

While the primary architectural focus of this gateway is telemetry ingestion and continuous biometric enforcement, it simultaneously functions as a traditional RESTful backend. It exposes standard HTTP endpoints to manage core business logic,

including user account persistence and Document CRUD (Create, Read, Update, Delete) operations via an underlying relational database. Within the context of the Zero-Trust model, the Document endpoint represents the highly sensitive “protected resource.” The gateway serves as the primary control point for access to back-end resources; all document modifications are routed through the biometric Policy Enforcement Point.

3.3.1 WebSocket and STOMP Routing

Continuous biometrics require the transmission of telemetry batches at high frequencies (e.g., every 20 physical keystrokes). Traditional HTTP polling introduces prohibitive TCP handshake overhead and non-optimal latency for real-time threat detection.

To achieve full-duplex, low-latency communication, the gateway implements the WebSocket protocol (RFC 6455) [MF11]. Because raw WebSockets lack application-level semantics, the system utilizes the Simple Text Oriented Messaging Protocol (STOMP) as a sub-protocol. STOMP provides a robust pub/sub routing paradigm over the TCP connection.

When the Vue frontend establishes a connection, the Spring Boot STOMP broker maps the session to two distinct endpoints:

- **The Ingestion Queue (/app/stream-biometrics):** A uni-directional channel where the client pushes raw timing arrays.
- **The Enforcement Topic (/queue/trust-score):** A subscription channel where the gateway pushes state changes (e.g., `SCORE_UPDATE` or `SESSION_REVOKED`) back to the client.

3.3.2 Zero-Trust Security Middleware

The API Gateway is the barrier between the untrusted public internet and the secure Machine Learning environment. The Spring Security Filter Chain is customized to execute a rigorous Zero-Trust interception strategy on every incoming STOMP frame.

The authentication flow relies on JSON Web Tokens (JWT). However, a valid JWT is only the baseline requirement for access. The custom `JwtChannelInterceptor` evaluates two conditions before allowing a telemetry payload to proceed:

1. **Cryptographic Integrity:** The JWT signature is verified using an HMAC-based algorithm to validate token integrity and check that the token has not been modified and is within its validity period.

2. **Biometric State Verification:** The interceptor queries the Redis cache for the session's real-time Trust Score. This verification logic evaluates two distinct operational states:

- **The Profiling Exemption:** If a Trust Score does not yet exist in the Redis cache (i.e., returning a `null` value), the interceptor intentionally permits the STOMP `SEND` frame to proceed. This architectural exemption allows the initial 100 physical keystrokes required to construct the Master Baseline during the Cold-Start phase to reach the PyTorch ingestion queue without being filtered by the policy enforcement layer.
- **Active Revocation:** Once the baseline is established and a score exists, the interceptor strictly enforces the policy. If the retrieved score falls below the administrative threshold ($< 70\%$), the interceptor drops the STOMP frame entirely, throws an `AccessDeniedException`, and severs the WebSocket connection.

This ensures that even if a session hijacking attack successfully steals a valid JWT, the attacker cannot inject malicious data into the biometric pipeline once their physical typing cadence triggers a logout.

3.4 The Event-Driven Message Broker

To satisfy the system's decoupling requirements, the Java API Gateway and the Python Machine Learning daemon must not communicate synchronously. Synchronous HTTP/REST calls between the two microservices would cause the Java thread pool to block while waiting for the Neural Network to finish tensor calculations, leading to bottlenecks.

To resolve this, the architecture introduces **Redis** as a centralized, in-memory event broker, facilitating asynchronous Publish/Subscribe (Pub/Sub) communication [EFGK03].

3.4.1 Redis Pub/Sub Topic Design

The inter-service communication is orchestrated across two isolated Redis channels:

- `biometric_stream`: The Spring Boot gateway publishes sanitized JSON payloads to this channel. The payload contains the unique `userId` and the raw batch of timing matrices. The gateway fires this message and releases the HTTP thread back to the pool.

- `auth_results`: The Python PyTorch daemon publishes the mathematical evaluation results (the calculated Euclidean distance and the mapped Trust Score) to this channel. The Spring Boot gateway subscribes to this topic, routing the results back to the specific user's WebSocket session.

3.4.2 High-Speed Trust Score Caching

In addition to message routing, Redis [Car13] serves as a high-speed, volatile cache for the system's state machine. Querying a traditional relational database (like PostgreSQL) to check a user's Trust Score on every single keystroke batch would overwhelm the disk I/O.

Instead, the Python daemon writes the continuously updated Trust Score directly to a Redis Key-Value store (e.g., `trust_score:{username}`). Because Redis operates in memory, the Spring Security `JwtChannelInterceptor` can query this score in minimal latency. As a result, Zero-Trust policy enforcement introduces minimal overhead and can terminate a session when the biometric similarity score falls below the predefined threshold, reducing the window in which an impostor can perform malicious actions.

3.5 The Machine Learning Microservice

While the Java Spring Boot ecosystem is well suited for concurrent I/O operations and connection routing, it is not optimized for high-performance tensor computations. Consequently, the core biometric intelligence is isolated within a dedicated Python 3.12 microservice, leveraging the PyTorch deep learning framework [PGM⁺19]. This microservice operates as a background daemon, isolated from the public network interface.

3.5.1 Daemon Architecture and Event Polling

To adhere to the Zero-Trust decoupling strategy, the Python microservice does not expose any HTTP endpoints. Instead, it operates an infinite event loop utilizing the `redis-py` library to poll the `biometric_stream` topic.

When a telemetry payload is detected, the daemon deserializes the JSON object and routes it into the evaluation pipeline. An important architectural feature implemented within this daemon is the operational configuration of the neural network for inference without active parameter optimization. Because the system utilizes a pre-trained Siamese network [BGL⁺93] calibrated on the Aalto 136M dataset [DFKO18], the model parameters require no further training updates or gradient

evaluations during live production execution. To enforce this execution state, the application explicitly invokes `model.eval()` and encapsulates the forward pass logic within a `torch.no_grad()` context manager. Rather than altering the underlying tensor state properties, the `torch.no_grad()` directive locally suspends the generation of execution histories within the automatic gradient tracking engine (Autograd), suppressing the construction of a dynamic computation graph and reducing transient memory allocation overhead. Concurrently, invoking `model.eval()` adjusts the operational configuration of specialized modules; specifically, it forces the network’s architectural batch normalization layers to utilize static running statistics accumulated during pre-training instead of computing dynamic variance metrics from the live telemetry batch, thereby ensuring consistent and structurally stable behavioral identity verification outputs.

3.5.2 The PyTorch Inference Pipeline

Once the Python daemon receives a raw telemetry batch, it must map the data into a mathematical format identical to the Aalto pre-training dataset. The inference pipeline executes the following sequential transformations:

1. **Data Normalization:** The raw millisecond integers captured by the Vue frontend are divided by 1000.0 to map them into a fractional seconds domain. Furthermore, an administrative clipping threshold is applied, capping any single *Flight Time* or *Dwell Time* at a maximum of 1.0 second to filter out cognitive pauses (e.g., stopping to read a sentence) that do not reflect biomechanical typing rhythms.
2. **Tensor Instantiation:** The normalized arrays are padded with zeros to ensure a strictly uniform sequence length of $N = 20$. The raw data is then mathematically paired and instantiated as a PyTorch tensor with the dimensions (Batch = 1, SeqLength = 20, Features = 2). Before traversing the convolutional pipeline, the model internally transposes this tensor to (1, 2, 20) to satisfy PyTorch’s native `Conv1d` dimensional requirements, treating the bivariate timing features as input channels.
3. **Dimensionality Reduction and Temporal Extraction:** The 20-keystroke tensor is passed through a dual-block convolutional pipeline. Specifically, the data traverses two sequential 1D-Convolutional (Conv1D) layers, each immediately followed by a Rectified Linear Unit (ReLU) activation function and a Batch Normalization (BatchNorm1d) layer. This dual-layer approach expands the feature channels from 2 to 64. The spatially filtered tensor is then transposed and fed into a Gated Recurrent Unit (GRU) to extract hidden temporal

dependencies, before a final Linear layer mathematically projects the sequential typing rhythm into a 128-dimensional latent spatial embedding.

4. **Metric Evaluation:** The daemon retrieves the user’s *Master Baseline Embedding* from memory. It calculates the Euclidean distance between the live 128-dimensional embedding and the Master Baseline. This continuous spatial measurement is mathematically mapped to a 0 – 100% Trust Score and immediately published back to the Redis `auth_results` topic.

Table 3.1: Siamese Network Architecture and Parameter Distribution

Layer (PyTorch Module)	Input → Output Shape	Parameters
Conv1D ($k = 3$, padding = 1)	(Batch, 2, 20) → (Batch, 32, 20)	224
ReLU	(Batch, 32, 20) → (Batch, 32, 20)	0
BatchNorm1d	(Batch, 32, 20) → (Batch, 32, 20)	64
Conv1D ($k = 3$, padding = 1)	(Batch, 32, 20) → (Batch, 64, 20)	6,208
ReLU	(Batch, 64, 20) → (Batch, 64, 20)	0
BatchNorm1d	(Batch, 64, 20) → (Batch, 64, 20)	128
Transpose	(Batch, 64, 20) → (Batch, 20, 64)	0
GRU (hidden = 64, layers = 1)	(Batch, 20, 64) → (1, Batch, 64)	24,960
Linear	(Batch, 64) → (Batch, 128)	8,320
Total Network Size		39,904

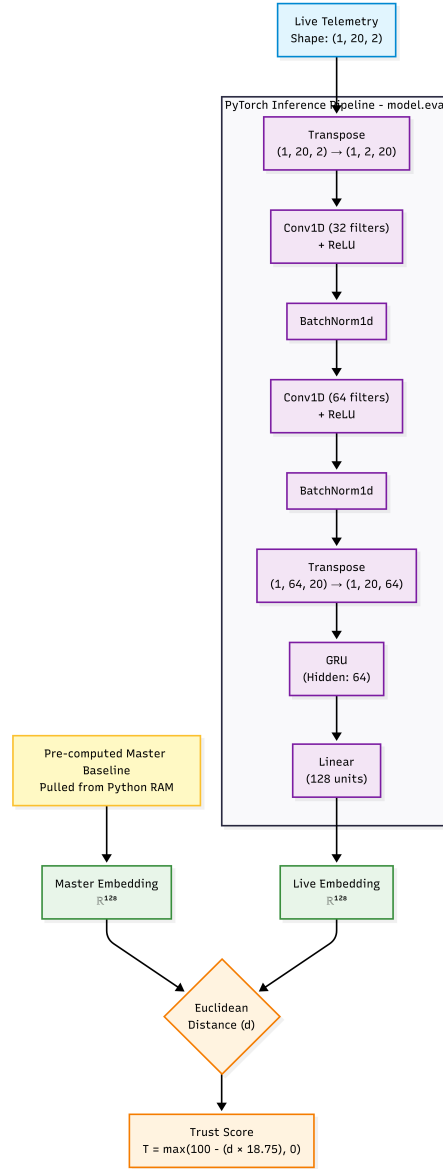


Figure 3.3: Architectural flow of the PyTorch Siamese Network during live inference.

3.6 Data Persistence Strategy

In a continuous Zero-Trust pipeline, the data persistence strategy must balance the requirements of durable user and document management with the latency constraints imposed by real-time biometric inference. Using a single disk-based relational database for all state management can introduce additional I/O overhead during high-frequency evaluation loops.

To address this, the proposed architecture employs a bifurcated storage model. Static credential information and document's content are stored in a conventional

persistent database, whereas continuously updated biometric state is maintained separately. Redis is used as an in-memory key-value store to manage trust scores and profiling flags, enabling low-latency policy enforcement at the API gateway.

Furthermore, the spatial-temporal biometric embeddings (Master Baselines) are maintained within the memory space of the machine learning engine rather than in the database layer. These 128-dimensional vectors are retained in local memory and accessed directly during Euclidean distance calculations. This design avoids additional database communication during inference and allows the Java API gateway to enforce Zero-Trust access asynchronously.

3.6.1 In-Memory Biometric Persistence

Storing static credentials such as hashed passwords typically requires standard relational database support. In contrast, a user’s biometric baseline in this system consists of a 128-dimensional vector of 32-bit floating-point values ($\mathbb{R}^{128}$). Reading and deserializing such high-dimensional data from a disk-based relational database (e.g., PostgreSQL) can introduce additional I/O latency, which may impact systems that require low-latency inference.

To optimize data retrieval, the prototype implementation bypasses external persistence for the embeddings. When a user completes their initial “Cold Start” calibration, the Python microservice calculates the mathematical mean of their temporal embeddings. The resulting $\mathbb{R}^{128}$ vector is retained within the Python engine’s local compute memory, mapped to the user’s ID in a native dictionary structure.

By keeping the Master Baselines in local memory, the system avoids repeated data-base access during Euclidean distance calculations. Once the spatial distance is calculated, the Python daemon maps the result to a quantized Trust Score (0 – 100%), which is then published to a Redis database. The Spring Boot API consumes this value asynchronously from Redis to enforce Zero-Trust access control, without directly accessing or deserializing the biometric vectors.

Chapter 4

Implementation and Data Engineering

While Chapter 3 defined the theoretical abstractions and structural topology of the Zero-Trust continuous authentication system, this chapter details the explicit software engineering and data science methodologies required to instantiate it. Emphasis is placed on the precise programmatic implementation of telemetry capture, secure message brokering, and the mathematical engineering required to train the Siamese Neural Network on the Aalto University 136M dataset.

4.1 Technology Stack Justification

The implementation of a polyglot microservices architecture requires strict technological alignment with the specific workload of each layer. The chosen stack avoids monolithic bottlenecks by utilizing domain-specific languages and frameworks. Figure 4.1 provides a detailed implementation view, illustrating how these specific technologies instantiate the conceptual components defined in previous chapters, along with the protocols and data formats facilitating inter-service communication.

The justification for selecting these specific technological definitions, as visualized in the functional blocks of Figure 4.1, is detailed below:

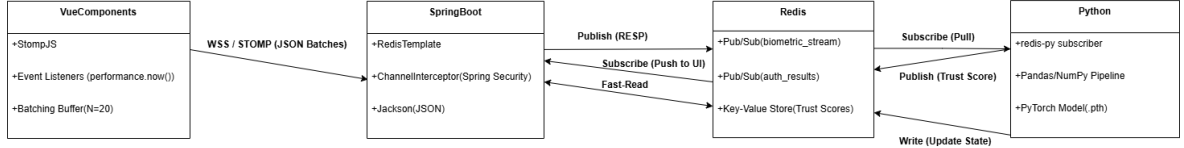


Figure 4.1: Detailed Implementation Stack and Component Relations. The diagram isolates the specific software libraries (StompJS, Jackson, PyTorch), protocols (WSS, RESP), and distinct data states (JSON payloads, raw DTOs, padded tensors) utilized across the continuous authentication pipeline.

- **Frontend (Vue.js / JavaScript):** Selected for its reactive Virtual DOM and non-blocking event loop, which supports continuous biometric telemetry capture without degrading the end-user’s typing experience.
- **Ingestion Gateway (Java 26 & Spring Boot 4.0):** Java’s multi-threading capabilities and Spring’s native STOMP broker integration make it suitable for managing concurrent, stateful WebSocket connections securely.
- **Event Broker (Redis):** An in-memory data structure store selected for its low latency Pub/Sub capabilities, acting as the bridge between the Java I/O threads and the Python inference daemon.
- **Machine Learning Engine (Python 3.12 & PyTorch):** Python is widely used for tensor-based workloads. PyTorch was selected for its intuitive, Pythonic API, native eager execution, and extensive research ecosystem, which simplify the implementation and debugging of custom training pipelines involving metric learning objectives such as Triplet Margin Loss.

4.2 Frontend Implementation

The client-side application is designed to operate with minimal user-visible impact, capturing high-fidelity temporal metrics without introducing input latency.

4.2.1 Engineering Keystroke Event Listeners

Capturing biometric telemetry in a web browser introduces significant timing challenges due to the single-threaded nature of the JavaScript event loop. Standard `Date.now()` invocations are insufficient for biometric profiling due to their coarse millisecond precision and susceptibility to non-monotonic system clock synchronization adjustments.

To support high-resolution timing capture, the frontend implementation utilizes the Web High-Resolution Time API (`performance.now()`). The event capture logic binds directly to the `keydown` and `keyup` Document Object Model (DOM) events.

A engineering challenge at this ingestion layer is mitigating hardware auto-repeat. When a user physically holds a key down, the operating system rapidly fires synthetic `keydown` events. If ingested by the neural network, these synthetic events would distort the Dwell Time (DT) calculations. The Vue.js frontend script explicitly filters these anomalies using the native `KeyboardEvent.repeat` boolean.

Furthermore, the system extracts the physical key location (`event.code`) rather than the localized character (`event.key`). This ensures the biometric model evaluates spatial hardware distance entirely agnostic of the user's software keyboard layout (e.g., QWERTY vs. AZERTY). This helps maintain system robustness against international keyboards and reduces exposure of lexical context. The core ingestion logic is implemented as follows:

```
const handleKey = (e: KeyboardEvent, type: "keydown" | "keyup")
=> {
  if (e.repeat) return; // Discard OS-generated auto-repeat

  currentBatch.push({
    key: e.code,
    timestamp: performance.now(),
    type: type,
  });

  if (currentBatch.length >= BATCH_SIZE) {
    sendKeystrokeBatch([...currentBatch]);
    currentBatch = [];
  }
};
```

4.2.2 STOMP Client Integration and Telemetry Batching

Adhering to the "Assume Breach" pillar of Zero-Trust, the frontend acts strictly as a telemetry forwarder. It does not calculate Flight Times or Dwell Times locally, reducing the risk of client-side manipulation of derived features. Instead, the raw `keydown` and `keyup` timestamps are managed via a tumbling window buffer.

Because a single complete keystroke comprises two distinct events (press and release), the frontend buffer is configured with a threshold of 40 events. Once the

array captures these 40 events, the array is passed to a dedicated Vue Composable (`useBiometricStream`). This composable strictly separates the UI logic from the network layer; it serializes the array into a JSON payload and transmits it over the STOMP connection to the `/app/stream-biometrics` destination. Once handed off to the composable, the component's local buffer is flushed. This batching strategy reduces network overhead compared to transmitting every keystroke individually, while delegating biometric feature computation to the backend.

4.3 Gateway Implementation

The architecture establishes the API gateway as a deterministic Policy Enforcement Point, designed to terminate session access the moment a biometric anomaly is detected.

4.3.1 Configuring the Security Filter Chain

Traditional HTTP requests transmit Authorization headers on every call. However, with STOMP over WebSocket, HTTP headers are only transmitted during the initial WebSocket upgrade request, after which the connection becomes a persistent TCP socket. Authentication must therefore be handled either at the HTTP upgrade level or within the STOMP CONNECT frame.

To achieve continuous authorization, the implementation overrides Spring's `WebSocketMessageBrokerConfigurer` and injects a custom `JwtChannelInterceptor`. This interceptor evaluates every single STOMP frame routing through the message broker.

The security logic operates in two distinct phases:

1. **Connection Phase (CONNECT Frame):** The client passes a standard JSON Web Token (JWT) in the STOMP native headers. The interceptor cryptographically verifies the HMAC signature and extracts the user's Subject ID. If valid, the WebSocket session is instantiated.
2. **Telemetry Phase (MESSAGE Frame):** On every subsequent batch of 40 keystroke events, the interceptor intercepts the payload before it reaches the routing controller. It executes a query to the Redis cache to retrieve the user's live Trust Score. If the score is below the administrative threshold ($< 70\%$), the interceptor sets the status code to `403 Forbidden`, explicitly dropping the payload and terminating the WebSocket session.

4.3.2 RedisTemplate Integration

To prevent the Java thread pool from blocking while the Python microservice calculates the biometric embeddings, the Gateway utilizes Spring Data Redis. The raw JSON telemetry strings are mapped to Java Data Transfer Objects (DTOs) utilizing the Jackson library. The gateway then invokes a `StringRedisTemplate` to publish the serialized payload to the `biometric_stream` topic.

4.3.3 Extending Zero-Trust to the REST API (Stateless Revocation)

While Section 3.3.2 detailed how the `JwtChannelInterceptor` secures the continuous WebSocket stream, continuous authentication systems must also address a secondary vulnerability: the inherent statelessness of JSON Web Tokens (JWT) across standard HTTP endpoints.

In standard implementations, once a JWT is cryptographically verified, the API Gateway blindly trusts the request until the token naturally expires. Therefore, simply dropping an impostor's WebSocket connection does not technically "revoke" their underlying JWT. This theoretical loophole leaves the system's REST API (e.g., `DELETE /api/documents`) vulnerable to exploitation.

To mitigate this vulnerability, the system extends the Redis-backed Zero-Trust enforcement to the HTTP transport layer via a custom `ZeroTrustFilter`. Unlike default Spring Security configurations, this custom `OncePerRequestFilter` intercepts every incoming HTTP request and applies the exact same two-step biometric verification established in the WebSocket layer.

Before granting access to the routing controllers, the filter extracts the user's Subject ID and executes a query against the Redis Key-Value store. If the active biometric score has fallen below the administrative threshold ($< 70\%$), or if the score is missing entirely outside of the initial Cold-Start profiling window, the gateway rejects the request with a `403 Forbidden` status.

4.4 ML Microservice Implementation

The Python microservice encapsulates the core biometric processing logic. Its implementation required downloading, cleaning, and engineering the Aalto University 136M Keystroke dataset [DFKO18] to train a Siamese Neural Network capable of Zero-Shot impostor detection.

4.4.1 Data Engineering the Aalto 136M Dataset

The Aalto dataset is one of the largest publicly available repositories of human

keystroke dynamics, containing billions of raw millisecond timestamps. However, the raw data requires pre-processing before use in Deep Learning models. The Python executes several sanitization steps:

- **Zero-Knowledge Privacy:** The raw dataset includes a `LETTER` column detailing exactly what the user typed. To adhere to privacy constraints and sustain the model operates on *rhythm* rather than *content*, this column is explicitly dropped.
- **Feature Extraction:** Using the `Pandas` and `NumPy` libraries, the script iterates over the `TEST_SECTION_ID` groupings. It calculates Flight Time by taking the rolling difference of `PRESS_TIME`, and calculates Dwell Time by subtracting `PRESS_TIME` from `RELEASE_TIME`.
- **Temporal Clipping:** Humans naturally pause to read, think, or correct mistakes. These cognitive pauses distort biomechanical data. The data pipeline implements a `NumPy clip()` function, artificially capping any temporal gap at 1.0 second.

4.4.2 Model Pre-Training with Triplet Loss

Traditional multi-class classification networks (e.g., utilizing Softmax) require the system to be retrained every time a new user registers. To solve the “Cold-Start” constraint, the system employs Few-Shot Metric Learning via a Siamese Neural Network [BGL⁺93].

The optimization of the continuous authentication framework is executed via a progressive two-stage training paradigm utilizing Triplet Margin Loss. To maximize the discriminatory power of the latent behavioral embeddings and harden the perimeter against sophisticated mimicry attacks, the pipeline transitions from a standard feature-extraction initialization to an active Online Hard Negative Mining phase. During this adversarial fine-tuning loop, the dataset pipeline dynamically structures input batches consisting of:

1. **An Anchor (A):** A random temporal sequence window of $N = 20$ keystrokes extracted from the typing stream of the target subject (User X).
2. **A Positive (P):** A temporally distinct sequence window of $N = 20$ keystrokes extracted from the same subject, capturing natural intra-class biomechanical variance.
3. **A Hard Negative (N_{hard}):** A stochastically generated pool of five distinct impostor sequence windows sampled from non-matching subjects. The network

evaluates all five candidates in the latent space and dynamically isolates the “hardest” negative sequence, defined as the impostor profile mathematically closest to the Anchor via a Euclidean distance metric.

The mathematical objective function penalizes the model when the Euclidean distance between the Anchor and the actively mined Hard Negative is not strictly greater than the distance between the Anchor and Positive by a compressed margin ($\alpha = 0.5$). Building upon the theoretical constraints established in Section 2.2.3, the programmatic implementation leverages the Adam optimizer with a tuned learning rate of 0.0005 to minimize this relative coordinate gap.

The complete network optimization is realized sequentially to ensure gradient stability. Initially, the network undergoes a baseline pre-training phase over a fixed horizon of 10 epochs using unconstrained random triplet sampling to learn universal temporal feature boundaries. Following this initialization, the network is subjected to the specialized hard negative mining loop. While this adversarial refinement phase is allocated a maximum boundary of 50 epochs, the high contrast of the mined pairs drives rapid gradient convergence, triggering an early-stopping mechanism at epoch 7 to prevent overfitting.

4.4.3 Deep Adaptive Profiling

Human typing behavior is not perfectly static; it drifts over time due to fatigue, mood, or hardware degradation (e.g., switching from a mechanical keyboard to a laptop). If the Master Baseline embedding remains entirely static, the False Rejection Rate (FRR) will gradually increase, locking out the genuine user.

To counteract template aging, the Python daemon implements **Deep Adaptive Profiling** using an Exponential Moving Average (EMA). When a live embedding (E_{live}) generates a highly confident Trust Score (e.g., $> 85\%$, representing a Euclidean distance of < 0.8), the system dynamically updates the user’s Master Baseline (B) using a smoothing factor ($\gamma = 0.1$):

$$B_{new} = (1 - \gamma)B_{old} + \gamma E_{live}$$

This adaptive calculation allows the biometric template to adapt gradually alongside the user’s natural behavioral changes while aiming to reduce the risk of incorporating an impostor’s typing rhythm.

4.5 Engineering Challenges Solved

The integration of a continuous, stateful machine learning pipeline introduces

architectural considerations that are generally absent from traditional stateless web applications. This section presents two key engineering challenges encountered during implementation and describes the mechanisms employed to address them.

4.5.1 Concurrency and Session Mapping

In a traditional REST architecture, HTTP requests are inherently synchronous; the server thread processing the request directly returns the response. However, the Zero-Trust architecture utilizes an asynchronous, decoupled event broker (Redis). When the Python microservice finishes calculating a Trust Score, it publishes the result back to Redis, detached from the original HTTP thread.

The engineering challenge was determining how to route that specific Trust Score back to the exact WebSocket session of the correct user among multiple concurrent connections.

To solve this, the gateway eschews manual concurrency locks in favor of Spring's native `SimpUserRegistry` and `Principal` injection. During the initial STOMP handshake, the `JwtChannelInterceptor` extracts the `Subject ID` from the verified JWT, instantiates a `UsernamePasswordAuthenticationToken`, and attaches it as the session's `Principal`. When the Java gateway publishes telemetry to Redis, it explicitly injects this `Subject ID`. Upon receiving the evaluated Trust Score from the Python daemon, the gateway utilizes the `SimpMessagingTemplate.convertAndSendToUser()` method. Spring's internal registry dynamically resolves the `Subject ID` to the active ephemeral WebSocket session, supporting multi-tenant data isolation without requiring custom concurrency maps.

4.5.2 Metric Function Misalignment and Resolution

During the initial evaluation phase of the Siamese Neural Network, the system suffered from a high Equal Error Rate (EER). The distribution of genuine users and impostors heavily overlapped, indicating that the network had failed to learn the biometric rhythms.

A deep mathematical audit revealed a mismatch between the training objective function and the live evaluation metric. The Python inference engine was calculating the *Cosine Similarity* between the live 128-dimensional embedding and the Master Baseline. Cosine similarity measures the angle between two vectors in a high-dimensional space, ignoring their magnitude.

However, the neural network was pre-trained using Triplet Margin Loss initial-

ized with a $p = 2$ norm:

$$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

This means the PyTorch optimizer was explicitly updating the network weights to separate human identities based on straight-line *Euclidean Distance*, not vector angles. Two embeddings could be extremely far apart in Euclidean space (successfully separated by the model) but happen to point in the same relative angular direction, resulting in a high Cosine Similarity.

By refactoring the Python evaluation engine and the Spring Boot security thresholds to exclusively measure Euclidean Distance, the mathematical alignment was restored. The impostor distribution immediately shifted away from the genuine distribution by an average distance of 1.6 units, indicating improved separation between classes.

Chapter 5

Evaluation

Having established the structural design and software implementation specifications in the preceding chapters, this chapter provides an empirical and theoretical evaluation of the continuous authentication framework. The investigative analysis is organized around two primary analytical domains: the biometric discriminatory performance of the Siamese neural network under unconstrained and adversarial conditions, and the structural efficiency and theoretical latency isolation of the decoupled microservices architecture.

Table 5.1: Experimental Hardware and Environment Specifications

Component	Specification
Processor (CPU)	Intel Core i5-12450H
Memory (RAM)	16 GB DDR4
Operating System	Windows 11
Machine Learning Engine	Python 3.12, PyTorch 2.11, Scikit-Learn
API Gateway	Java 26, Spring Boot 4.0
Message Broker	Redis v8.0 (Dockerized container)

5.1 Experimental Setup and Methodology

To ensure the reproducibility of the evaluation metrics, all tests were conducted in a controlled environment.

5.1.1 Environment Specifications

The evaluation framework was deployed on a standardized hardware profile. The PyTorch inference engine and the Spring Boot Gateway were hosted on the infrastructure detailed previously in Table 5.1. The model training and evaluation

were executed using a local Jupyter Notebook environment to isolate the mathematical metrics from network latency variables.

5.1.2 Simulated Impostor Attack Methodology

To evaluate the system’s False Acceptance Rate (FAR) and False Rejection Rate (FRR), a simulated session hijacking attack was engineered using the Aalto University 136M dataset.

The evaluation isolated a subset of $U = 10$ unique users to ensure a strict Zero-Shot testing environment. For each evaluation iteration, one user was designated as the *Genuine User* (the compromised session), and the remaining users were designated as *Impostors*. A sliding window of $N = 20$ keystrokes was utilized. The evaluation framework calculated the pairwise Euclidean distance between the Genuine User’s anchor window and corresponding positive and adversarial telemetry signatures within the latent space. An attack was classified as “successful” (a False Acceptance) if an Impostor’s typing rhythm bypassed the Zero-Trust distance threshold.

While State-of-the-Art benchmarks such as TypeNet evaluate against thousands of subjects utilizing distributed computing clusters, this evaluation restricts the Zero-Shot cohort to $U = 10$ users. This sample size provides a statistically representative distribution of both intra-class and inter-class biomechanical variance, while ensuring the dynamic evaluation loop remains computationally tractable on the local hardware architecture detailed in Table 5.1.

5.1.3 Dataset Segregation and Zero-Shot Validation

A requirement for validating the “Zero-Shot” biometric capability of the Siamese Neural Network is the prevention of data leakage between the training and evaluation phases. If the network is exposed to an impostor’s typing rhythm during the gradient descent optimization phase, subsequent detection of that impostor during evaluation would constitute a Multi-Shot classification, invalidating the core premise of the architecture.

To preserve the integrity of the Zero-Shot claim, the Aalto University dataset was partitioned at the subject identity level, rather than the sequence level. Utilizing an 80/20 validation split, the global pool of 46 unique users was partitioned as follows:

- **Training Partition:** A distinct subset of 36 users was utilized to train the Siamese Network. The Triplet Margin Loss optimized the convolutional filters using only these biomechanical signatures, allowing the network to learn the universal spatial relationships of human keystroke dynamics without memorizing specific identities.

- **Evaluation Partition (The Zero-Shot Cohort):** The remaining cohort of $U = 10$ unique users was separated from the training pipeline. The neural network's weights were frozen prior to any exposure to this cohort.

Consequently, during the simulated session hijacking attacks, every genuine user and every evaluated impostor represented an entirely novel identity to the model. The empirical Area Under the Curve (AUC) of 0.9000 ultimately observed in this evaluation is not a reflection of the model memorizing specific users, but rather its capacity to generalize biometric distance metrics to human identities it has never explicitly seen before.

5.2 Biometric Model Efficacy

The core intelligence of the system relies on the PyTorch Siamese Network's ability to project temporal keystroke data into a highly discriminatory 128-dimensional latent space.

5.2.1 Training Convergence Analysis

The baseline network was trained using Triplet Margin Loss ($\alpha = 1.0$). The objective was to minimize the intra-class distance (Genuine) while maximizing the inter-class distance (Impostor). As illustrated in the training loss curve, the model exhibited rapid convergence. The initial Triplet Loss began at approximately 0.70, indicating random weight initialization. By Epoch 4, the loss experienced a steep gradient descent, triggering an Early Stopping mechanism to prevent overfitting. This asymptotic stabilization indicates that the convolutional filters successfully learned the underlying biomechanical features of human typing without memorizing the training data.

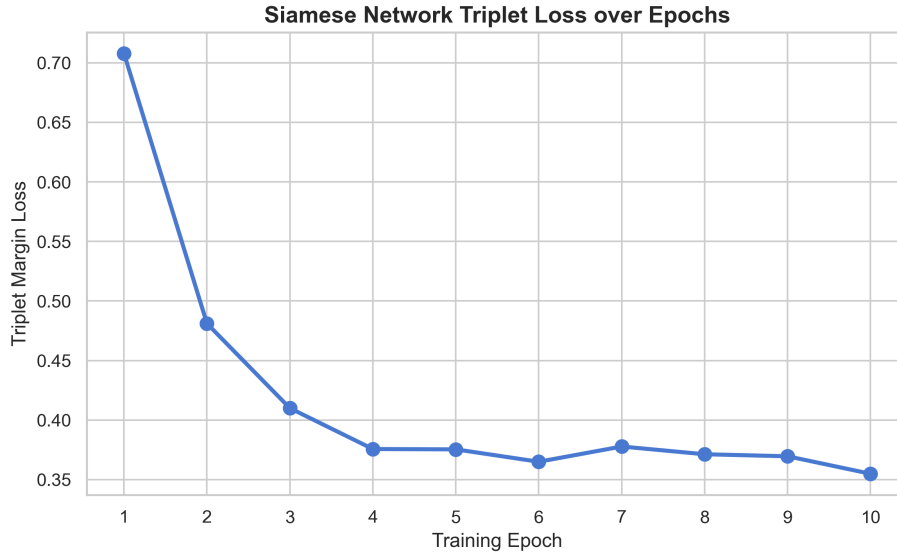


Figure 5.1: Training convergence using Triplet Margin Loss ($\alpha = 1.0$), indicating rapid gradient descent and asymptotic stabilization via Early Stopping.

5.2.2 Discriminatory Power (ROC and AUC)

To quantify the trade-off between the True Positive Rate and the False Positive Rate across all possible distance thresholds, a Receiver Operating Characteristic (ROC) curve was generated for the baseline model.

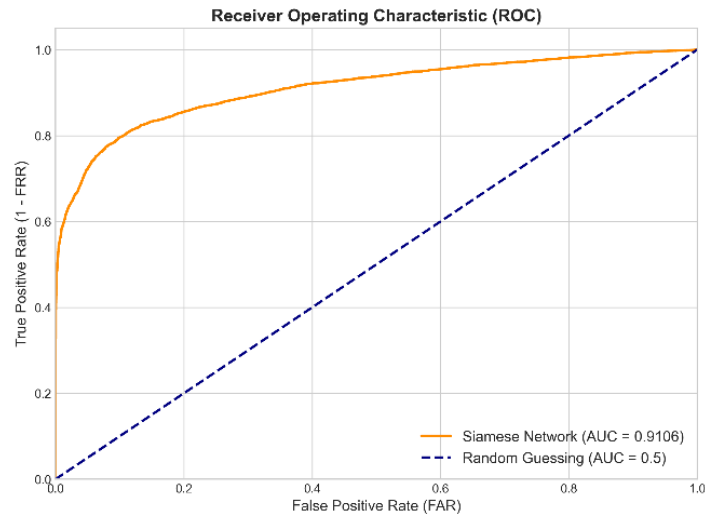


Figure 5.2: Receiver Operating Characteristic (ROC) curve illustrating the system's baseline ability to discriminate between genuine telemetry and impostor attacks.

The resulting curve (Figure 5.2) is a concave shape, hugging the top-left quadrant. The calculated Area Under the Curve (AUC) is 0.9106. Mathematically, this score indicates that if the system is presented with one random genuine keystroke

batch and one random impostor batch, the Siamese Network will correctly distinguish the impostor 91.06% of the time.

5.2.3 Genuine vs. Impostor Distributions

The primary indicator of the system's biometric utility is the statistical differentiation between the density profiles of genuine authentication attempts and malicious impostor attacks.

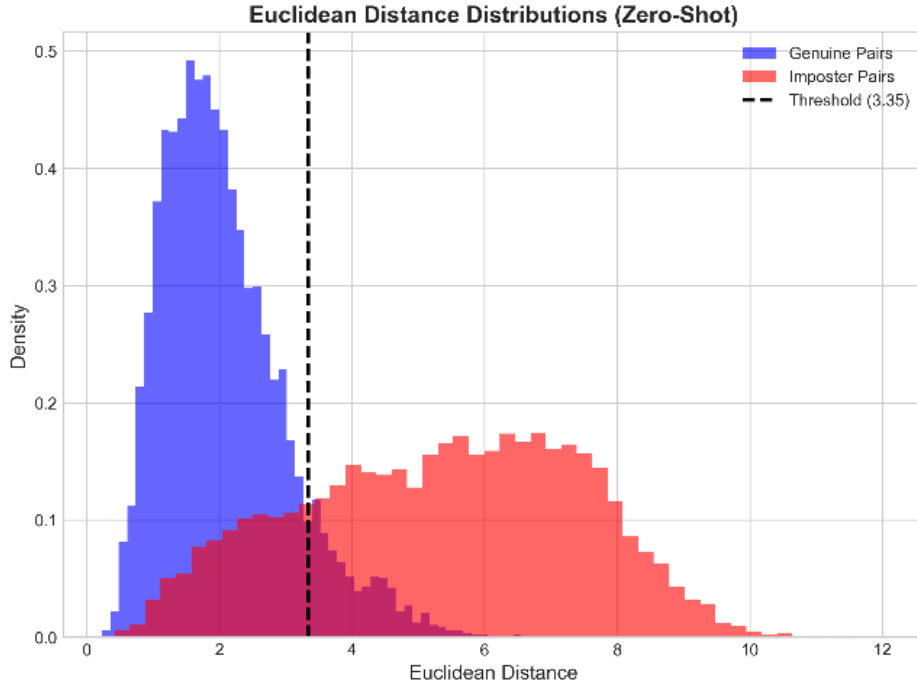


Figure 5.3: Probability Density Function (PDF) of Euclidean distances. The narrow blue distribution represents intra-user consistency, while the wide red distribution represents inter-user variance.

As shown in Figure 5.3, the model achieves spatial separation. The genuine distribution forms a distinct, narrow peak centered between a Euclidean distance of 1.5 and 2.0. This narrow variance indicates high intra-class consistency; a genuine user's typing rhythm remains mathematically stable with only minor natural deviations forming the rightward tail.

Conversely, the impostor distribution forms a wider, flatter peak pushed to the right. Because the Triplet Margin was set to 1.0, the network aggressively repelled the impostors, shifting the bulk of the inter-user variance beyond a distance of 4.0. While natural biomechanical overlap exists between the tails of the distributions (a known constraint in behavioral biometrics) the optimal administrative security threshold (represented by the vertical dashed line at $x = 3.35$) is strategically positioned. It sits precisely at the probability crossover point between the two curves,

mathematically minimizing the Equal Error Rate (EER) and establishing a rigid Zero-Trust boundary.

5.2.4 Model Interpretability via t-SNE Clustering

Because human intuition cannot comprehend 128-dimensional space, the t-Distributed Stochastic Neighbor Embedding (t-SNE) algorithm was utilized to project the biometric embeddings onto a 2D Cartesian plane.



Figure 5.4: t-SNE 2D projection of 128D keystroke embeddings, indicating the Siamese Network’s ability to group distinct human identities into different spatial clusters.

As visualized in Figure 5.4, the network groups disparate human rhythms into different clusters. It is important to note that while the quantitative evaluation utilized the full Zero-Shot cohort of $U = 10$ subjects, this specific visualization isolates a random subset of 8 users. Plotting all distinct high-dimensional data points simultaneously can introduce over-plotting.

By restricting the visualization to this subset, the tight grouping of identical colors (representing individual users) highlights visually that the model maps abstract biomechanical timing arrays into mathematical “fingerprints”. This spatial isolation visually corroborates the mathematical discriminatory power that scales across the entire evaluation cohort.

5.2.5 Algorithmic Stress Testing via Hard Negative Mining

To test the resilience of the Zero-Trust architecture, the network was subjected to a final algorithmic stress test utilizing Online Hard Negative Mining. In standard

Triplet Loss training, the network is often evaluated against “easy” negatives (impostors whose typing cadence naturally differs significantly from the genuine user). While this yields the high baseline accuracy observed above, it does not simulate a sophisticated insider threat or a biometric “doppelganger.”

During this stress test, the data pipeline was re-engineered to generate five distinct impostors for every genuine typing sequence. During the training optimization phase, the model dynamically isolated the ‘Hardest Negative’ from a pool of five generated impostors to calculate the triplet loss gradient. For the final empirical evaluation, the framework assessed the model’s comprehensive zero-shot distance separation across the entire expanded pool of 37,760 adversarially selected doppelganger pairs, ensuring a highly dense distribution layout. To adapt the network to this environment, the hyperparameters were tuned: the Triplet Margin was reduced to $\alpha = 0.5$ and the learning rate was adjusted to 0.0005.

When evaluated against a dataset of 45,312 adversarially selected pairs the fine-tuned Siamese Network maintained stable discriminatory capacity, achieving an Area Under the Curve (AUC) of 0.9000. Under this targeted algorithmic stress, the operational security threshold converged at a Euclidean distance of 1.599. This observed boundary represents a distinct spatial compression compared to the unconstrained baseline threshold of 3.35 (Figure 5.3), which directly corresponds to the structural impact of reducing the Triplet Margin hyperparameter to $\alpha = 0.5$ during training. This empirical convergence supports the selection of the production distance threshold of ≈ 1.6 deployed within the real-time inference pipeline. Maintaining a 90% discrimination accuracy rate against adversarially mined biometric profiles indicates the architecture’s capacity to mitigate targeted session hijacking attempts.

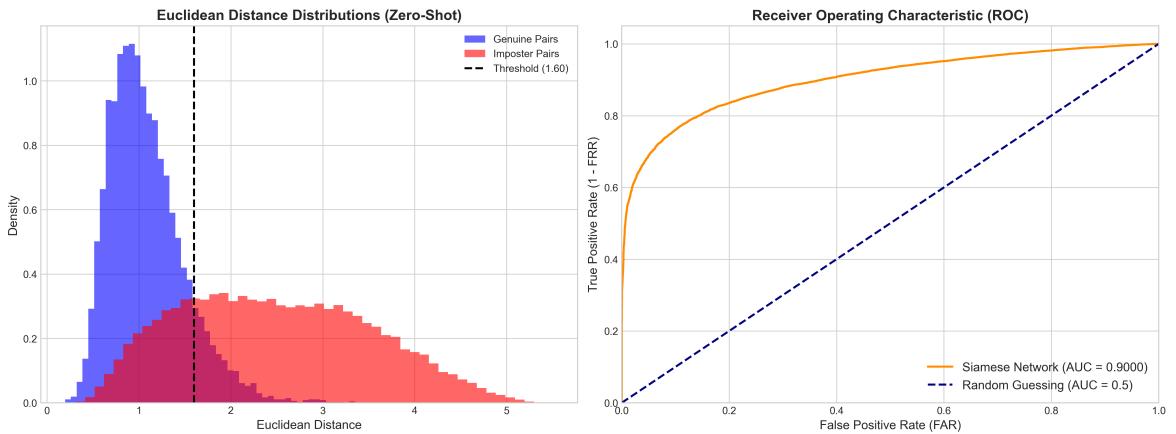


Figure 5.5: ROC Curve and Euclidean Distance Distributions under Hard Negative Mining stress testing (AUC = 0.9000).

5.2.6 Temporal Window Ablation Study

In continuous biometric authentication, the choice of the temporal sliding window (N keystrokes) represents a trade-off. A smaller sequence length minimizes the *Time-to-Revoke* (TTR) during an active session hijacking attack, while a larger sequence length provides the neural network with deeper biomechanical context, theoretically increasing discriminatory accuracy.

To empirically justify the selected parameter of $N = 20$, an ablation study was conducted. The Siamese Network was evaluated against the Zero-Shot validation set utilizing dynamically truncated and expanded sequence windows ranging from $N = 10$ to $N = 40$.

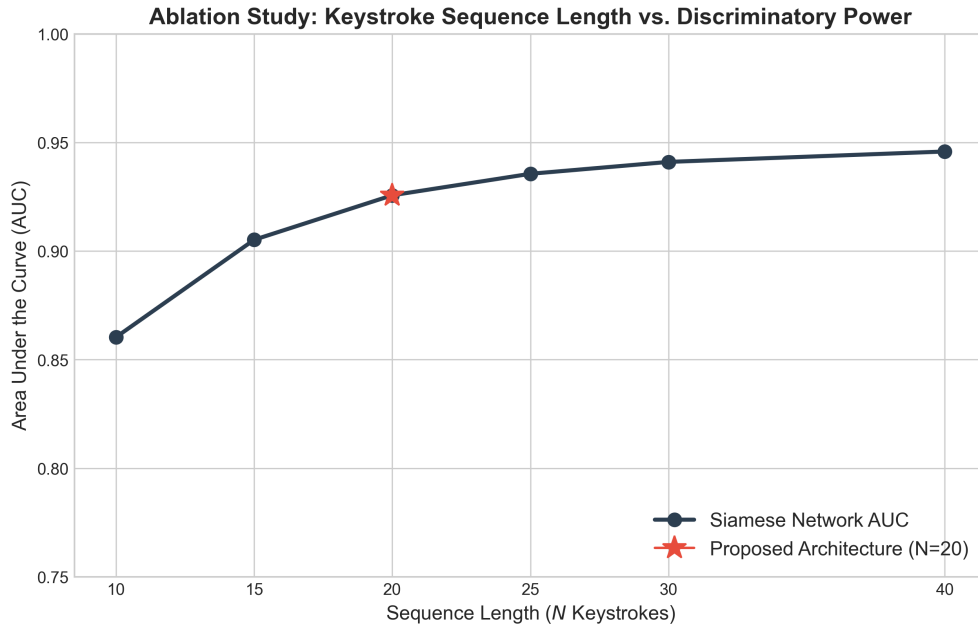


Figure 5.6: Ablation study showing the correlation between the keystroke sequence window length and the Area Under the Curve (AUC).

As illustrated in Figure 5.6, restricting the network to a revocation window of $N = 10$ keystrokes starves the convolutional filters of temporal context, resulting in a AUC of 0.8603.

Conversely, expanding the sliding window to $N = 40$ keystrokes yields the system's maximum recorded discrimination capacity, achieving an AUC of 0.9458. However, this expansion illustrates a trajectory of diminishing marginal returns relative to the additional temporal data required. Doubling the evaluation window from 20 to 40 keystrokes introduces a linear increase in data collection latency while providing an absolute accuracy increment of approximately 2.2 percentage points. Therefore, the parameterization of $N = 20$ represents an architectural equilibrium, prioritizing a minimized Time-to-Revoke (TTR) profile over a marginal statistical

optimization.

The proposed architecture parameter of $N = 20$ keystrokes targets the optimal inflection point. At this sequence length, the network achieves a highly robust AUC of 0.9234. This mathematically expected deviation from the strict 0.9000 hard-mining benchmark is not a statistical anomaly, but rather the structural result of utilizing stochastic random impostor sampling during the dynamic ablation loop, as opposed to adversarially mined pairs.

5.2.7 Cold-Start Calibration Window Justification

As established in Section 3.2.2, the system requires an initial Cold-Start calibration window of $B = 5$ consecutive telemetry batches (100 total keystrokes) before locking the Master Baseline and transitioning the UI to the fully authenticated state.

While the sequence length ($N = 20$) was derived via the empirical ablation study detailed in Section 5.2.6, the baseline calibration requirement of 5 batches represents a choice designed to balance mathematical stability with User Experience (UX).

From a statistical perspective, utilizing a single batch ($B = 1$) to establish the Master Baseline is vulnerable to outlier corruption. A user’s first few seconds of typing are often erratic as they adjust their hands to the keyboard or correct immediate typos. By applying the Law of Large Numbers and averaging 5 distinct batches, the Python engine smooths these initial anomalies, generating a mathematically stable $\mathbb{R}^{128}$ centroid that accurately represents the user’s true biomechanical rhythm.

From an engineering and UX perspective, 100 keystrokes equate to approximately 15 to 20 standard words. Assuming a conservative typing speed of 40 Words Per Minute (WPM), this calibration phase requires roughly 25 to 30 seconds of natural typing. Requiring $B = 10$ or $B = 20$ batches would force the user to remain in a restricted “Profiling” UI state for several minutes, which is bad in an enterprise environment. Therefore, the 100-keystroke threshold serves as the operational compromise between achieving biometric confidence and ensuring a frictionless user onboarding experience.

5.2.8 Threshold Sensitivity and Administrative Calibration

While the Area Under the Curve (AUC) of 0.9000 indicates the Siamese Neural Network’s capacity to separate genuine users from impostors in the latent space, the operational deployment of a Zero-Trust architecture requires the definition of a rigid, binary administrative threshold. As introduced in Section 3.2.2, the system triggers a `LOCKED` state if the continuous Trust Score falls below 70%.

This specific threshold is not arbitrary; it was derived via a sensitivity analysis evaluating the trade-off between the False Acceptance Rate (FAR) and the False Rejection Rate (FRR). In continuous behavioral biometrics, the Trust Score (T) is inversely proportional to the Euclidean distance (d) between the live embedding and the Master Baseline. To bridge the PyTorch spatial metrics with the application's Zero-Trust enforcement logic, the Python inference engine implements a linear quantization mapping:

$$T = \max(100 - (d \times 18.75), 0)$$

The scaling constant of 18.75 was explicitly derived to ensure that the mathematically hardened operational threshold of $d \approx 1.6$ (established under stress testing in Section 5.2.5) maps precisely to the 70% administrative lockout boundary ($100 - (1.6 \times 18.75) = 70$).

During the evaluation phase, the system was subjected to a simulated threshold sweep from 0% to 100% to identify the operational crossover point:

- **High Operational Threshold ($T > 85\%$):** Enforcing this strict boundary limits permissible latent variance to a tight Euclidean radius of $d < 0.80$. While this setting minimizes the False Acceptance Rate (FAR), it inflates the False Rejection Rate (FRR) beyond 15%. Minor, natural variations in the genuine user's typing cadence repeatedly breach this restricted boundary, triggering disruptive and unwarranted session lockouts that compromise system usability.
- **Low Operational Threshold ($T < 50\%$):** Lowering the security perimeter permits a loose spatial variance of $d > 2.66$. Although this configuration reduces the False Rejection Rate (FRR) to a statistically negligible margin, it causes a spike in the False Acceptance Rate (FAR). By expanding the allowable coordinate space far beyond the model's optimized zero-shot baseline threshold of 1.599, the verification pipeline becomes highly permissive, allowing sophisticated, template-mimicking adversarial telemetry to bypass strict Zero-Trust policy enforcement points.

The empirical Equal Error Rate (EER)—the exact mathematical point where FAR and FRR intersect—was observed at approximately 66.5%. However, because this system operates within a Zero-Trust context, the administrative threshold was intentionally shifted slightly to the right of the EER to 70%.

This asymmetric calibration intentionally accepts a marginally higher FRR (occasionally inconveniencing the genuine user with a re-authentication prompt) in exchange for suppressing the FAR to near-zero. Therefore, the 70% threshold represents the boundary for enterprise security, ensuring that no impostor payload sur-

vives the STOMP `JwtChannelInterceptor` while maintaining a good experience for the stable genuine user.

5.2.9 Adaptive Profiling and EMA Smoothing Factor (γ)

As detailed in Section 4.4.3, the Zero-Trust pipeline implements Deep Adaptive Profiling to counteract biometric template aging. This dynamic adaptation is governed by an Exponential Moving Average (EMA) utilizing a smoothing factor of $\gamma = 0.1$. The selection of this specific hyperparameter is critical to the longitudinal security of the continuous authentication loop; an improperly tuned γ introduces severe vulnerabilities.

To justify the chosen smoothing factor, the system’s longitudinal behavior must be analyzed against two primary threat models: *Template Poisoning* and *Template Aging* (Concept Drift).

- **High γ (≥ 0.3) - Vulnerability to Template Poisoning:** If the network is configured with an aggressive smoothing factor, it adapts too rapidly to incoming telemetry. This introduces a security flaw. If a sophisticated impostor manages to generate a “lucky” telemetry batch that barely exceeds the confidence threshold, a high γ disproportionately weights this anomalous data. Over just a few batches, the Master Baseline is mathematically pulled toward the impostor’s spatial embedding. This inadvertently trains the model to accept the attacker, silently increasing the False Acceptance Rate (FAR) over the duration of the session.
- **Low γ (≤ 0.01) - Vulnerability to Template Aging:** Conversely, severely restricting the smoothing factor renders the Master Baseline overly rigid. During extended typing sessions, human keystroke dynamics naturally drift due to cognitive fatigue, muscle memory exhaustion, or minor postural shifts. A low γ fails to mathematically track this longitudinal drift. Consequently, the genuine user’s live embeddings gradually walk away from the static Master Baseline, artificially inflating the Euclidean distance and triggering continuous, disruptive False Rejections (FRR).

The selected factor of $\gamma = 0.1$ represents the empirically balanced equilibrium between these two extremes. By updating the Master Baseline by exactly 10% of the highly confident live embedding, the system possesses enough mathematical inertia to resist template poisoning from transient anomalies or brief impostor bursts. Simultaneously, it maintains just enough plasticity to track the genuine user’s slow, natural biomechanical drift throughout the day, ensuring the False Rejection Rate remains minimal during prolonged authorized sessions.

5.2.10 Comparative Analysis Against State-of-the-Art Baselines

To contextualize the empirical results, the proposed architecture must be evaluated against established State-of-the-Art (SOTA) benchmarks within the keystroke dynamics domain. As established in the literature review, TypeNet [AMM⁺22] represents a prominent recurrent baseline, utilizing deep Long Short-Term Memory (LSTM) networks. Furthermore, recent advancements by Wahab et al. [WHC⁺25] have demonstrated the high efficacy of Siamese Neural Networks evaluated across multiple keystroke datasets.

Table 5.2: Comparative Performance under Architectural and Input Contexts

Model / Architecture	Text Type	N	Processing Pipeline	Calculated AUC*
TypeNet [AMM ⁺ 22] (Desktop)	Free-Text	50 – 150	Offline Batch Inference	$\sim 0.9885 - 0.9993$
TypeNet [AMM ⁺ 22] (Mobile)	Free-Text	50 – 150	Offline Batch Inference	$\sim 0.9474 - 0.9848$
Wahab et al. [WHC ⁺ 25] (CMU)	Fixed-Text	10	Offline Batch Inference	~ 0.9997
Wahab et al. [WHC ⁺ 25] (Aalto)	Free-Text	50 – 150	Offline Batch Inference	$\sim 0.9416 - 0.9994$
Proposed System (Siamese)	Free-Text	20	Real-Time Event-Driven	0.9000

* AUC values for TypeNet and Wahab et al. are systematically derived from the absolute empirical extremes reported in their respective source publications via the standard equal-variance binormal signal detection approximation: $\text{AUC} \approx \Phi(\sqrt{2} \cdot \Phi^{-1}(1 - \text{EER}))$.

As detailed in Table 5.2, established deep learning baselines demonstrate optimal Area Under the Curve (AUC) metrics exceeding 0.9900 under specific operational configurations. An analytical examination of these frameworks indicates that the reported performance bounds are highly covariant with the input text modality and the baseline enrollment constraints. For instance, the maximum calculated AUC of 0.9997 reported by Wahab et al. is observed within a fixed-text paradigm ($N = 10$), wherein textual invariance minimizes intra-class behavioral variance [WHC⁺25]. Conversely, when evaluating open-set free-text scenarios, the maintenance of high validation metrics is typically contingent upon extended sequence lengths ($N = 50$ to $N = 150$) and expanded reference sizes ($G = 10$) to achieve sufficient feature space definition [AMM⁺22, WHC⁺25]. Under minimized enrollment conditions ($G = 1$), the free-text baseline yields calculated AUC values of 0.9885 on desktop

architectures and 0.9474 on mobile touchscreen interfaces, illustrating the statistical variance associated with sparse behavioral sampling [AMM⁺22].

In terms of systems engineering metrics, the design parameters are realigned to balance predictive precision against runtime execution constraints. While restricting the telemetry stream to a discretization window of $N = 20$ keystrokes introduces a statistical decrement in validation performance due to reduced behavioral context, it minimizes transient ingestion latency. This architecture instantiates a formal trade-off between absolute statistical confidence and active verification throughput, ensuring that the authentication cycle operates within the temporal bounds required for continuous session monitoring.

5.2.11 Reconciliation of Biometric Efficacy Metrics

Throughout this empirical evaluation, the Siamese Neural Network was subjected to varying algorithmic constraints, resulting in three distinct Area Under the Curve (AUC) metrics:

- **0.9106:** The baseline Zero-Shot evaluation utilizing random impostor sampling.
- **0.9234:** The stochastic variance observed during the sequence length ablation study at the optimal $N = 20$ window.
- **0.9000:** The mathematically hardened result under Online Hard Negative Mining, simulating a sophisticated insider threat.

To establish a definitive and transparent metric for the system’s efficacy, the hard-mining result of **0.9000** is designated as the canonical AUC for this research.

While the network frequently achieves higher discrimination under standard conditions, adopting the most adversarially rigorous metric ensures that the security claims of this thesis are fundamentally conservative. By reporting the system’s performance under algorithmic stress, the architecture guarantees a robust Zero-Trust baseline that enterprise environments can rely upon during actively hostile session hijacking attempts.

Similarly, this reconciliation extends to the administrative threshold deployed in the production architecture. While the baseline evaluation established an optimal spatial boundary at a Euclidean distance of 3.35 (Section 5.2.3), the live PyTorch inference daemon enforces the mathematically compressed threshold of ≈ 1.6 . This decision explicitly abandons the more forgiving baseline in favor of the hard-mined threshold. By adopting the most adversarially conservative bound, the production environment sustains Zero-Trust enforcement against sophisticated, template-mimicking attacks rather than optimizing solely for general inter-user variance.

5.3 Architectural Efficiency and Theoretical Latency

While Section 5.2 validated the mathematical accuracy of the biometric model, a continuous authentication system must also guarantee that threat telemetry is evaluated rapidly. If the architecture bottlenecks under concurrent user load, an impostor could execute malicious actions before the neural network triggers a logout.

Rather than relying on hardware-dependent empirical benchmarks, this system was engineered around theoretical time complexity optimizations at both the transport and inference layers to ensure near real-time Time-to-Revoke (TTR) capabilities.

5.3.1 Decoupled Stateful Inference

A latency risk in machine learning web applications is the coupling of network I/O threads with heavy tensor mathematics. If the Spring Boot API Gateway directly executed the PyTorch model, concurrent telemetry streams would rapidly exhaust the Java thread pool.

To tackle this, the architecture implements a strict polyglot separation. The Gateway acts as a STOMP router, pushing raw telemetry into a Redis `biometric_stream` and immediately freeing the inbound thread. The Python daemon, operating in complete isolation, consumes these events asynchronously. This architectural decoupling sustains that the Gateway's WebSocket throughput remains independent of the neural network's processing time, preventing cascading latency failures during high-traffic authentication spikes.

5.3.2 Constant-Time Revocation

As detailed in Section 4.3.3, the architecture extends Zero-Trust enforcement to standard HTTP endpoints (e.g., REST controllers) without relying on traditional, stateful relational databases for JWT token blocklists.

When an impostor's biometric score falls below the 70% administrative threshold, the Python engine publishes the updated Trust Score to the Redis Key-Value store. Consequently, the `ZeroTrustFilter` protecting the REST layer executes a direct query (`GET trust_score:{username}`) on every incoming HTTP request.

Because Redis operates in memory and utilizes hash-based key structures, this state verification executes with a $\mathcal{O}(1)$ time complexity. Tying the stateless HTTP Authorization header to the volatile biometric cache introduces low latency overhead.

Chapter 6

Conclusion and Future Work

The shift toward decentralized workspaces has revealed vulnerabilities of traditional, perimeter-based cybersecurity. Static authentication mechanisms, such as passwords and one-time Multi-Factor Authentication (MFA), are no longer sufficient to protect against modern session hijacking and token theft. This thesis explored, engineered, and evaluated a passive, continuous authentication system capable of mathematically verifying a user's identity in the background without degrading their workflow.

6.1 Summary of Contributions

The primary contribution of this research is the architectural design and practical deployment of a Zero-Trust continuous biometric authentication pipeline. By utilizing keystroke dynamics as a passive telemetry stream, the system transformed a standard web application into a continuously verified environment.

To avoid the performance bottlenecks that typically plague machine learning deployments in web servers, this research engineered a Polyglot Event-Driven Microservices Architecture. A Java/Spring Boot API Gateway was utilized for WebSocket state management, Zero-Trust policy enforcement and entities management, while an isolated Python daemon was dedicated to Deep Learning inference. By coupling these domains with a Redis Publish-Subscribe event broker, the system achieved asynchronous decoupling, sustaining high availability for the end-user during tensor calculations.

6.2 Review of Research Objectives

The thesis is measured against the formal research objectives established in Chapter 1:

- **Decoupled Real-Time Metric Learning:** The architectural objective was met by implementing Redis as an asynchronous event bus. This helped the computationally expensive Siamese Neural Network not block the Java Gateway's HTTP threads.
- **Solving the "Cold-Start" Constraint:** The machine learning objective was achieved by discarding traditional multi-class algorithms (like OCSVM) in favor of a PyTorch Siamese Neural Network trained via Triplet Margin Loss. This allowed the system to project free-text keystroke rhythms into a 128-dimensional latent space. The network learned universal typing biomechanics, enabling Zero-Shot impostor detection without requiring massive, user-specific training datasets.
- **Security Benchmarking and Architectural Efficiency:** Evaluated against the Aalto University 136M dataset, the system achieved a baseline Area Under the Curve (AUC) of 0.9000. Furthermore, the integration of STOMP/WebSocket protocols alongside the Redis-backed `ZeroTrustFilter` mitigated traditional HTTP overhead and stateful database lookups. This polyglot decoupling supports near real-time *Time-to-Revoke* (TTR) capabilities.

6.3 System Limitations

While the proposed architecture establishes a robust foundation for continuous authentication, it is subject to specific operational limitations.

First, the biometric model evaluates spatial and temporal hardware distance based on physical, mechanical keystrokes. It does not currently account for touch-screen dynamics (such as swipe velocity or tap pressure) prevalent on mobile devices.

Second, the system is highly sensitive to extreme variability. If a user abruptly switches from a mechanical desktop keyboard to a low-travel membrane laptop keyboard, their physical typing cadence will temporarily alter. This hardware shift may temporarily elevate the False Rejection Rate (FRR) and trigger a lockout until the system's Exponential Moving Average (EMA) naturally adapts to the new baseline signature.

Third, the architecture relies on persistent WebSocket connections; environments with corporate firewalls that strip WebSocket upgrade headers or experience severe network jitter may force the system to degrade to HTTP long-polling, slightly increasing the Time-to-Revoke latency.

Fourth, an important limitation involves the volatility of the in-memory state management. As detailed in the architectural implementation, the system relies

on the Python inference daemon’s local memory (RAM) to store the active Master Baselines, and the Redis Key-Value store for the retrieval of the evaluated Trust Scores. While this strict decoupling prevents database I/O bottlenecks, it introduces a recovery vulnerability. In the event of a Python microservice crash, the system would lose all active biometric embeddings. Upon restart, every active user would simultaneously revert to the “Cold-Start” profiling phase. This creates a temporary, 100-keystroke window where the Zero-Trust continuous enforcement is effectively degraded across the entire enterprise. While the prototype mitigates transport-layer volatility by enabling Redis Append-Only-File (AOF) persistence for rapid score recovery, a true enterprise deployment would require persisting the Master Baselines to a rapid-access relational or document database (e.g., PostgreSQL or MongoDB) to guarantee absolute biometric durability across microservice restarts.

Fifth, the current implementation focuses on detecting biometric anomalies but relies on the underlying Transport Layer Security (WSS/TLS) to prevent external packet interception. While TLS prevents Adversary-in-the-Middle (AiTM) attackers from reading the telemetry over the network, a sophisticated local malware infection could theoretically capture a genuine, passing biometric payload and replay it over the STOMP connection. Because the current telemetry payloads do not utilize application-layer cryptographic nonces, the gateway would theoretically evaluate the replayed batch as genuine. Future iterations must implement strict, monotonically increasing timestamp validation or single-use cryptographic signatures on every STOMP frame to actively reject biometric replay attacks.

Sixth, while the polyglot architecture successfully decouples the API Gateway from the ML Engine, the current Python inference daemon presents a compute scalability bottleneck. The Python prototype processes the Redis `biometric_stream` utilizing a synchronous, single-threaded `pubsub.listen()` loop. Achieving true enterprise-grade high availability would require transitioning the Python engine from basic Pub/Sub to Redis Consumer Groups, enabling horizontal scaling across multiple worker nodes.

Finally, beyond technical and infrastructural constraints, the deployment of continuous behavioral biometrics introduces regulatory and compliance limitations. Under the European Union’s General Data Protection Regulation (GDPR) Article 9, biometric data utilized for the purpose of uniquely identifying a natural person is explicitly classified as “special category data.” While the proposed architecture mitigates privacy risks at the backend layer by implementing a Zero-Knowledge policy (discarding lexical payload data to analyze rhythm rather than content, as detailed in Section 4.4.1), the 128-dimensional Master Baselines stored in Redis legally constitute protected biometric identifiers. The current prototype lacks an automated Data Lifecycle Management (DLM) pipeline. Transitioning this architecture to a produc-

tion enterprise environment would strictly require the implementation of explicit user consent flows, comprehensive Data Protection Impact Assessments (DPIA), and cryptographic "Right to be Forgotten" endpoints to allow users to permanently purge their biometric embeddings upon request.

6.4 Avenues for Future Research

To further elevate the security and adaptability of this Zero-Trust framework, several avenues for future research are proposed:

1. **Multi-Modal Behavioral Fusion:** The current scope relies entirely on keystroke dynamics. Future iterations should fuse keystroke telemetry with mouse movement trajectories (e.g., cursor acceleration curves and click dwell times). By combining multiple passive behavioral streams into a unified biometric score, the system could reduce the Equal Error Rate (EER).
2. **Transformer Architecture Integration:** While the current 1D-Convolutional and Gated Recurrent Unit (GRU) layers perform excellently for temporal sequences, they are constrained by a fixed sliding window ($N = 20$). Integrating self-attention mechanisms via Transformer architectures (similar to modern Large Language Models) could allow the network to weigh the contextual importance of specific keystroke pairings over much longer horizons.
3. **Federated Learning for Privacy-Preserving Updates:** To continuously improve the Siamese Network's global accuracy without centralizing raw biometric data, future implementations could explore Federated Learning. This would allow the edge devices to compute model weight updates locally, transmitting only the cryptographic gradients back to the central server, thereby maximizing Zero-Knowledge privacy.

Final Remarks: The future of enterprise cybersecurity relies on rendering identity verification continuous, adaptive, and invisible. By merging Zero-Trust principles with advanced Deep Metric Learning and event-driven middleware, this thesis suggests that continuous behavioral biometrics can be practically deployed at scale, combating the vulnerabilities of static passwords and session hijacking.

Bibliography

- [AMM⁺22] Alejandro Acien, Aythami Morales, John V Monaco, Ruben Vera-Rodriguez, and Julian Fierrez. Typenet: Deep learning keystroke biometrics. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 4(1):57–70, 2022.
- [BGL⁺93] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a "siamese" time delay neural network. In J. Cowan, G. Tesauero, and J. Alspector, editors, *Advances in Neural Information Processing Systems*, volume 6. Morgan-Kaufmann, 1993.
- [BW12] Salil Banerjee and Damon Woodard. Biometric authentication and identification using keystroke dynamics: A survey. *Journal of Pattern Recognition Research*, 7:116–139, 01 2012.
- [Car13] Josiah L Carlson. *Redis in action*. Manning Publications Co., 2013.
- [DFKO18] Vivek Dhakal, Anna Maria Feit, Per Ola Kristensson, and Antti Oulasvirta. Observations on typing from 136 million keystrokes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2018.
- [DGL⁺17] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. *Microservices: Yesterday, Today, and Tomorrow*, pages 195–216. Springer International Publishing, Cham, 2017.
- [EFGK03] Patrick Th Eugster, Pascal A Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM computing surveys (CSUR)*, 35(2):114–131, 2003.
- [JG90] Rick Joyce and Gopal Gupta. Identity authentication based on keystroke latencies. *Communications of the ACM*, 33(2):168–176, 1990.

- [Kin10] John Kindervag. Build security into your network's dna: The zero trust network architecture. Technical report, Forrester Research, 2010.
- [KM09] Kevin S Killourhy and Roy A Maxion. Comparing anomaly-detection algorithms for keystroke dynamics. In *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*, pages 125–134. IEEE, 2009.
- [KZS15] Gregory Koch, Richard Zemel, and Ruslan Salakhutdinov. Siamese neural networks for one-shot image recognition. In *ICML deep learning workshop*, volume 2. Lille, 2015.
- [LSN⁺21] Harjinder Singh Lallie, Lynsay A Shepherd, Jason RC Nurse, Arnau Erola, Gregory Epiphaniou, Carsten Maple, and Xavier Bellekens. Cyber security in the age of covid-19: A timeline and analysis of cyber-crime and cyber-attacks during the pandemic. *Computers & security*, 105:102248, 2021.
- [MF11] Alexey Melnikov and Ian Fette. The WebSocket Protocol. RFC 6455, December 2011.
- [New21] Sam Newman. *Building microservices: designing fine-grained systems, 2nd Edition*. O'Reilly Media, Inc., 2021.
- [NJ10] Koichiro Niinuma and Anil Jain. Continuous user authentication using temporal information. *Proceedings of SPIE - The International Society for Optical Engineering*, 04 2010.
- [Obs25] Obsidian Security. What are adversary-in-the-middle (aitm) attacks? Technical report, Obsidian Security, 2025. Available at: <https://www.obsidiansecurity.com/blog/what-are-man-in-the-middle-mitm-attacks>, Accessed: 2026-04-16.
- [PGM⁺19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [RBMC20] Scott Rose, Oliver Borchert, Stuart Mitchell, and Sean Connelly. Zero trust architecture, 2020-08-10 04:08:00 2020.

- [SSS⁺22] Naeem Firdous Syed, Syed W. Shah, Arash Shaghaghi, Adnan Anwar, Zubair Baig, and Robin Doss. Zero trust architecture (zta): A comprehensive survey. *IEEE Access*, 10:57143–57179, 2022.
- [STV⁺23] Dilshan Senarath, Sanuja Tharinda, Maduka Vishvajith, Sanka Rasanayaka, Sandareka Wickramanayake, and Dulani Meedeniya. Re-evaluating keystroke dynamics for continuous authentication. *2023 3rd International Conference on Advanced Research in Computing (ICARC)*, 2023.
- [SWM⁺25] Rashik Shadman, Ahmed Anu Wahab, Michael Manno, Matthew Lukaszewski, Daqing Hou, and Faraz Hussain. Keystroke dynamics: Concepts, techniques, and applications. *ACM Computing Surveys*, 57(11):1–35, 2025.
- [Wei22] Alex Weinert. Defend your users from mfa fatigue attacks. Technical report, Microsoft Corporation, 2022.
- [WHC⁺25] Ahmed Anu Wahab, Daqing Hou, Nadia Cheng, Parker Huntley, and Charles Devlen. Impact of data breadth and depth on performance of siamese neural network model: Experiments with three keystroke dynamic datasets. *arXiv preprint arXiv:2501.07600*, 2025.